



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Runtime Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Student ID: s344024

Company Tutor:

Dr. David Soldani

Academic Year 2025-2026

An extended Berkeley Packet Filter (eBPF)-based tool was developed to monitor process activity on Rocky Linux 8.10. The work focuses on kernel hooks that capture process creation and image replacement, credential changes, inter-process interactions, and operations involving files, memory, namespaces and cgroups.

The monitoring logic combines syscalls, tracepoints, kernel functions and Linux Security Module interfaces. Because these attachment points observe different stages of kernel execution, the collected events can represent a process request, a security decision, a committed state change or the final result of an operation. Event data are stored in a bounded binary format, transferred through a perf-event channel and decoded by a Go user-space component.

Probe loading is driven by the active configuration so that only the required instrumentation is enabled. Decoded events are then evaluated through policies and deterministic detectors. Policies restrict the monitored scope, while detectors identify either single events or short ordered sequences of related activity. Alerts preserve the events that triggered them and may be associated with MITRE ATT&CK metadata.

The prototype was evaluated in a single-node Minikube environment using an adapted Capture the Flag challenge. During the experiment, the monitor captured process activity generated from a separate pod and reconstructed the sequence leading to shell execution and access to a protected file. The evaluation demonstrates the feasibility of using eBPF to obtain detailed and configurable process-level visibility while keeping event collection close to the relevant kernel execution paths.

Acknowledgements

Only in the printed version

Contents

Acronyms	IX
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	2
1.3 Objectives and Thesis Structure	3
2 eBPF Architecture	5
2.1 Linux Kernel Observability	5
2.2 eBPF Fundamentals	8
2.2.1 Execution Model, Verifier and JIT Compilation	8
2.2.2 Object Files, Loading and Attachment Lifecycle	10
2.2.3 Hook Selection and Event Semantics	11
2.2.4 Maps, Helpers and Kernel-to-Userspace Transport	12
2.2.5 BTF and CO-RE Portability	14
2.3 Related eBPF Runtime Security Tools	15
2.3.1 Tracee	15
2.3.2 Other Relevant eBPF Security Tools	16
2.3.3 Comparative Summary	17
2.4 Positioning of the Proposed Work	18

3	Requirements and System Design	19
3.1	Design Goals and Constraints	19
3.1.1	Target Environment and Compatibility Assumptions .	20
3.1.2	Functional Requirements	20
3.1.3	Non-Functional and Operational Requirements	22
3.2	System Architecture	23
3.2.1	Kernel-Space and User-Space Responsibilities	24
3.2.2	End-to-End Event Lifecycle	25
3.3	Kernel-Space Collection Design	25
3.3.1	Hook Selection and Logical Events	25
3.3.2	Early Event Selection and Kernel-Side Filtering	26
3.4	Event Contract and Transport Design	27
3.4.1	Stable Binary Event Contract	27
3.4.2	Perf-Buffer Transport and Failure Boundaries	29
3.5	User-Space Runtime Design	29
3.5.1	Loading, Attachment and Runtime Lifecycle	30
3.5.2	Decoding, Normalisation and Event Registration	30
3.5.3	Event Output, Alert Output and Operational Logging	31
3.6	Policy and Detection Architecture	31
3.6.1	Policy-Based Event Scope	31
3.6.2	Point and Collective Detectors	32
3.6.3	Bounded Local Correlation	32
3.6.4	Alert Evidence and MITRE ATT&CK Metadata	33
3.7	Design Decisions and Scope Boundaries	33
4	Kernel-Space Event Collection Implementation	35
4.1	Build Toolchain and Project Organisation	35
4.2	eBPF Program Implementation	36
4.2.1	Shared Event-Construction Framework	36

CONTENTS

4.2.2	Process Lifecycle and Executable Transitions	38
4.2.2.1	Process Creation and Lifecycle	38
4.2.2.2	Executable Transitions	39
4.2.3	Identity, Credentials, Process Control and Limits . . .	41
4.2.3.1	Identity and Credentials	41
4.2.3.2	Process Control and Limits	43
4.2.4	File and Filesystem Security	45
4.2.5	Memory, Namespace and Cgroup Transitions	48
4.2.6	Kernel Integrity and Kernel-Facing Activity	51
4.3	Binary Event Contract and Kernel Transport	53
4.4	Kernel-Side UID Filtering	55
4.5	Compatibility and Verifier Challenges	55
4.6	Kernel-Space Implementation Summary	56
5	User-Space Runtime and Detection Implementation	57
5.1	Command-Line Configuration and Runtime Bootstrap	57
5.2	Loader, Probe Registry and Attachment Lifecycle	58
5.3	User-Space Decoder and Event Registry	60
5.4	User-Space Event Selection and Admission	61
5.5	Policy and Detector Engine	62
5.5.1	YAML Policy and Detector Definitions	62
5.5.2	Point and Collective Evaluation Paths	64
5.5.3	Compiled Correlation State and Alert Deduplication .	65
5.6	Output and Structured Logging	68
6	Experimental Evaluation	69
6.1	Evaluation Objectives and Experimental Protocol	69
6.2	Kubernetes and Minikube as the Evaluation Platform	71
6.3	Cross-Pod Minikube Experimental Testbed	72

CONTENTS

6.4	AES-beta Workload and Controlled Attack Scenario	74
6.5	Functional Evaluation	75
6.5.1	Collective Detector Validation	75
6.5.2	Benign Control and Repeated Runs	78
6.6	Performance Evaluation	79
6.6.1	Metrics, Sampling and Benchmark Profiles	79
6.6.2	Monitor CPU and Memory Consumption	80
6.6.3	Effect of Kernel-Side UID Filtering	81
6.7	Discussion of Results	81
6.8	Threats to Validity and Current Limitations	82
7	Conclusions and Future Work	83
7.1	Summary of the Work	83
7.2	Main Results	84
7.3	Limitations	85
7.4	Future Work	85
A	Selected eBPF Process-Monitoring Source Code	87
A.1	Process Creation	87
A.2	Executable Transitions	88
A.3	Credential Transitions	90
	Bibliography	92

List of Figures

2.1	High-level eBPF execution workflow	9
2.2	Complementary evidence exposed by different observation points along a kernel operation.	12
2.3	Tracee high-level architecture	15
3.1	Kernel space and userspace architecture	24
3.2	Interpretation of an indexed event record	28
3.3	User-space runtime lifecycle and failure boundaries	30
4.1	Execution monitoring: attempts and outcomes	40
4.2	Real, Effective, Saved and Filesystem UIDs	41
4.3	Memory and isolation transition monitoring	49
5.1	Binary record layout consumed by the user-space decoder	60
6.1	Minikube local Kubernetes environment	72
6.2	Cross-Pod experimental topology	74
6.3	AES-beta exploitation and monitored evidence flow	75

List of Tables

2.1	Observation sources for eBPF runtime monitoring	7
2.2	Comparison of eBPF event-transport mechanisms	14
2.3	Architectural summary of the runtime security tools considered in this chapter.	18
3.1	Functional requirements of the monitoring system.	21
3.2	Compatibility and resource requirements of the monitor.	22
3.3	Distinction between physical hooks, producer and support probes, and logical events.	26
3.4	Illustrative monitoring plan for execution and file access.	31
5.1	Runtime bounds for collective state and alert deduplication.	67
5.2	User-space output and diagnostic boundaries.	68
6.1	Recorded environment and component boundaries of the cross-Pod testbed.	73
6.2	Application and detector outcomes of the repeated functional runs.	78
6.3	Per-run resource measurements for the monitor-container cgroup.	80

Acronyms

eBPF	Extended Berkeley Packet Filter
CPU	Central processing unit
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
LSM	Linux Security Module
JIT	Just-in-time
BTF	BPF Type Format
CO-RE	Compile Once - Run Everywhere
ELF	Executable and Linkable Format
API	Application programming interface
CLI	Command-line interface
CGO	C language interoperability mechanism for Go
JSON	JavaScript Object Notation
YAML	Yet Another Markup Language
PID	Process identifier
TID	Thread identifier
UID	User identifier

Acronyms

GID	Group identifier
VMA	Virtual memory area
IPC	Inter-process communication
UTS	UNIX time-sharing system
KABI	Kernel application binary interface
RHEL	Red Hat Enterprise Linux
BPF	Berkeley Packet Filter
CTF	Capture the Flag
RSS	Resident set size
SSH	Secure Shell
UTC	Coordinated Universal Time
SD	Standard deviation
RAM	Random-access memory
TOTP	Time-based one-time password

Chapter 1

Introduction

1.1 Context and Motivation

Applications, containers and infrastructure services continually change the state of a running Linux system. Static inspection describes software before it runs, and application logs record only what their developers chose to expose. Important operations may therefore remain outside both views. Process creation, credential changes and file access, for example, are mediated by the operating system whether or not an application reports them. Runtime security monitoring needs access to this lower-level activity while it is taking place.

The Linux kernel provides such an observation layer because it mediates access to processes, files, credentials, memory and other shared resources. Its telemetry can expose activity that spans several applications or never appears in their logs. This matters in containerised environments: workloads may be isolated from one another, but their operations still pass through the same host kernel. Instrumenting that kernel is itself security-sensitive, however. Excessive privileges or unsafe extensions can enlarge the attack surface, particularly across container boundaries [18]. Useful visibility must therefore be weighed against compatibility, runtime cost and the risks of privileged execution.

Runtime security tools increasingly rely on eBPF for this purpose. Existing systems use it to collect process, file, network and security telemetry in cloud-native environments, with different architectural and performance trade-offs [19]. The work presented here concentrates on process activity

and host security.

eBPF provides a controlled execution environment inside the Linux kernel. Programs loaded from user space can be attached to supported tracing, networking and security hooks. Before accepting one, the kernel verifier uses static analysis to examine its control flow and its handling of memory, registers, pointers and helper functions. Once accepted, the program runs whenever its attachment point is reached and can collect or transform the available data [16, 20, 21].

This model allows collection to occur where the kernel handles an operation, with less dependence on application-specific instrumentation. Maps and event buffers provide a structured path to user space, and unwanted activity can sometimes be discarded before transport. These facilities make eBPF a useful basis for runtime monitoring, but they do not guarantee safety, portability or low overhead. Verifier behaviour, program types, helpers, data structures and attachment mechanisms vary between kernel versions, so the monitoring pipeline must still be designed for its target system.

This work was carried out as part of an industrial collaboration and targets Rocky Linux 8.10. Its enterprise kernel includes vendor backports, so compatibility is treated as a design constraint[25].

1.2 Problem Statement

The problem considered in this thesis is how to turn kernel observations into useful information for security analysis on an enterprise Linux system. Analysts need to identify which processes performed an operation, which resources were involved and what happened. The information available depends on where the operation is observed. For example, a process may try to open a file and be denied access. An observation of the attempt tells us what the process tried to do; an observation of the result tells us whether it succeeded.

Understanding an individual operation may also require information about the activity surrounding it. Access to a sensitive file, for example, can be investigated more precisely when the analyst knows which program accessed it and how that program was started. The observations must retain enough information to connect these actions and attribute them to the processes involved.

A running host produces a continuous stream of activity from applications, background services and administrative tasks. The information relevant to an investigation depends on its purpose. Analysts therefore need to select the activity of interest while retaining the details required to understand it. Large volumes of unrelated observations make this task harder and increase the cost of collecting and processing the data.

The monitor must also work with the features available on the target system. Linux kernels differ in the operations they expose for monitoring and the information that can be collected. This work targets Rocky Linux 8.10, where the eBPF programs must pass the kernel's checks before they can run. Monitoring also uses CPU and memory, which it shares with the applications being observed.

The central problem is therefore to provide a coherent view of security-relevant host activity from observations that expose different parts of an operation. This view must allow analysts to identify the processes and resources involved, distinguish attempts from outcomes and focus on the activity relevant to their investigation. Achieving this requires sufficiently detailed evidence, reliable interpretation and collection compatible with the monitored system.

1.3 Objectives and Thesis Structure

The objective of this thesis is to design, implement and evaluate an eBPF-based runtime security monitor for the target Rocky Linux environment. The prototype collects process and security-related kernel events, normalises them in user space and evaluates them through configurable policies and local detectors.

The remaining chapters cover the technical background, system design, implementation and experimental evaluation:

- **Chapter 2 - eBPF Architecture** introduces Linux runtime instrumentation, the eBPF execution model, object loading and attachment, maps, transport and portability. It also reviews related eBPF-based runtime-security tools.
- **Chapter 3 - Requirements and System Design** defines the target environment, functional and operational requirements, architectural

components, event flow and the boundaries between kernel-space collection and user-space processing.

- **Chapter 4 - Kernel-Space Event Collection Implementation** describes the shared eBPF event framework, process and security hook families, binary record contract, perf-buffer transport, early filtering and compatibility work required by the target kernel.
- **Chapter 5 - User-Space and Detection Implementation** presents the Go runtime, probe registry, attachment lifecycle, decoder, event selection, policy and detector subsystems, MITRE ATT&CK metadata, output handling and structured logging.
- **Chapter 6 - Experimental Evaluation** evaluates the prototype in a Kubernetes environment deployed with Minikube. The experimental scenario uses a vulnerable CTF workload as the target of controlled attacks to assess event collection, detector operation and alert generation, together with CPU and memory consumption measured within the monitor's cgroup.
- **Chapter 7 - Conclusions** summarises the results, discusses the principal limitations of the prototype and identifies possible directions for future development.

Chapter 2

eBPF Architecture

2.1 Linux Kernel Observability

Application logs provide valuable domain-specific information, but only for actions that the application was designed to record. The kernel provides a different view. Process creation, credential changes, file access, memory management and inter-process communication all pass through it, including operations that span applications or are absent from their logs. Observing these operations while they occur supplies the evidence needed for runtime monitoring [50, 16].

There is no universal kernel event source. Linux instead exposes instrumentation points at several stages of execution, each with its own timing, context and dependence on implementation details.

System-call observation. A system call is the controlled interface through which a user-space process requests a service from the kernel. It is not itself an instrumentation mechanism. Monitoring systems commonly observe this boundary through system-call tracepoints, such as entry and exit events. An entry event exposes the requested operation and its original arguments, whereas an exit event exposes the returned result. Correlating the two can distinguish an attempted operation from one that completed successfully. This view is broad and process-oriented, but it remains close to the user-space request: one system call may traverse several internal functions, and its original arguments may not identify the final kernel object or security decision [55, 50].

Tracepoints and raw tracepoints. A tracepoint is a static observation point deliberately inserted into the kernel source by kernel developers. It has a name, belongs to an event category, and defines the data made available when that point is reached. A tracepoint remains inactive until a tracing component is attached to it. In an eBPF-based monitor, this component is an eBPF program loaded from user space and attached to the selected tracepoint. Each time kernel execution reaches that point, the kernel invokes the attached program and provides a tracepoint-specific context containing the declared event fields.

A regular tracepoint supplies a structured context whose layout follows the format exported for that event. A raw tracepoint uses the same underlying instrumentation site but exposes its original kernel arguments with less intermediate transformation. The latter reduces preparation before the eBPF program runs, at the cost of requiring kernel-aware argument decoding. Both forms remain limited to the tracepoints provided by the target kernel [55, 53].

Kprobes and kretprobes. A kprobe is a dynamic probe placed at a selected kernel instruction at run time. Monitoring tools usually identify that instruction through the symbol of a kernel function and place the probe at its entry. Unlike a tracepoint, the observation point does not need to have been prepared explicitly by the kernel developer. When execution reaches the probed instruction, the kernel temporarily transfers control to the probing infrastructure, which invokes the attached eBPF program before normal execution continues. The program receives a snapshot of the processor-register state. Function arguments are not presented as a tracepoint-specific event structure; they must be recovered from the appropriate registers according to the architecture and calling convention.

A kretprobe observes the corresponding function-return path. It causes an attached program to run when an invocation of the probed function finishes and makes the return value available through the register context. A kprobe can therefore describe the operation requested at function entry, while a kretprobe can reveal the result produced at function exit. Entry and return records are separate observations: when both the original arguments and the result are required in one logical event, the monitoring system must save the entry data and correlate it with the return observation, commonly by using the current thread identity.

Kprobes and kretprobes reach functions for which no tracepoint exists, but this flexibility increases coupling to the kernel implementation. A renamed

or inlined function, a changed signature, a different calling convention or an unavailable symbol can invalidate either attachment or argument decoding. Dynamic probes consequently require more target-specific validation than tracepoints [47].

Linux Security Module hooks. The LSM framework inserts security mediation points into kernel operations involving files, credentials, processes, sockets, and other protected objects. At these points, registered security modules can inspect an operation and, for hooks with authorisation semantics, permit or reject it. An LSM-related function can expose a resolved kernel object at a security-relevant stage of an operation, rather than only the original system-call request. Here, the hook and the attached program are distinct: the LSM hook is the kernel mediation point, while an eBPF LSM program is one means of attaching logic to a supported hook. If that program type is unavailable or unsuitable, a monitor may observe the corresponding security function with a kprobe. Entry-side observation alone does not necessarily reveal the final outcome; a compatible return-side probe is needed when that result matters [49, 53, 47].

Observation source	Information exposed	Main trade-off
System-call tracepoint	Arguments at entry or return value at exit.	Broad process visibility, but limited insight into internal kernel processing and resolved objects.
Kernel tracepoint	Structured fields defined for a specific kernel event.	Clear semantics and simple decoding, but only where a suitable tracepoint is available.
Raw tracepoint	Original tracepoint arguments with minimal preprocessing.	More direct access, but requires kernel-aware argument decoding.
Kprobe / kretprobe	Register state at function entry or return.	Flexible placement, but stronger dependence on kernel symbols, signatures and calling conventions.
LSM-related hook	Security context, resolved objects and mediation stage.	Rich security semantics, but attachment and result visibility depend on target-kernel support.

Table 2.1: Comparison of the principal observation sources relevant to eBPF-based runtime monitoring [55, 47, 49, 53].

The role of eBPF. eBPF is the execution facility attached to an observa-

tion point, not another name for the point itself. The hook determines when the program runs and which context it receives. The program can then extract fields, filter an observation, update a map or submit an event to user space. Thus, “tracepoint”, “kprobe”, “LSM hook” and “eBPF program” refer to different parts of the monitoring path [20, 53].

Choosing among these sources depends on the evidence required, the facilities available on the target kernel and the acceptable coupling to kernel internals. In practice, a monitor often combines them because no single source exposes every stage of an operation [50, 19].

2.2 eBPF Fundamentals

eBPF allows programs loaded from user space to run inside the Linux kernel when a selected event occurs. Each program is constrained by its type and attachment point, and the kernel verifies it before execution [27, 20].

This mechanism extends kernel behaviour without requiring a source change or a traditional module for every use case. Programs can be loaded, replaced and specialised for observability, performance analysis, networking or security. Their value lies in placing purpose-built logic close to the relevant kernel operation while retaining a defined execution model and a verification step [16, 21].

2.2.1 Execution Model, Verifier and JIT Compilation

An eBPF program is compiled into instructions for the BPF virtual machine and submitted to the kernel through the BPF user-space interface. The instruction set defines general-purpose registers, a bounded stack, arithmetic and branch instructions, memory operations, and calls to kernel-provided functions. The program does not execute as unrestricted kernel code: its available context, memory accesses, and callable functions depend on its program type [27, 22].

Code running in the kernel can affect the stability and security of the whole system, even when it was loaded from user space. The eBPF verifier is the kernel component that decides whether a submitted program may run. It analyses the program during loading and rejects it if the required safety conditions cannot be established.

The verifier performs a path-sensitive static analysis of the program’s control flow. It follows the possible execution paths while tracking the state of registers and stack locations, distinguishing scalar values from different classes of pointers, checking memory bounds and alignment, and validating the arguments supplied to helper functions. Paths that may access invalid memory, use uninitialised values, or leave kernel-managed resources unreleased are rejected. To avoid analysing equivalent situations repeatedly, the verifier uses state pruning: when a previously accepted verifier state already covers a new state reached at the same instruction, the redundant path does not need to be explored again [21, 17].

Verification establishes a set of safety properties, not the correctness of the monitoring logic. An accepted program may still read the wrong field, emit an incomplete event or disagree with its user-space decoder. Conversely, valid source-level logic may be rejected when a particular kernel cannot prove its safety. Verifier behaviour is thus both a security boundary and a compatibility constraint [21, 40].

After verification, an architecture-specific JIT compiler may translate the eBPF bytecode into native instructions. The kernel can then execute that code directly whenever the program is invoked instead of interpreting each eBPF instruction in turn [45, 46].

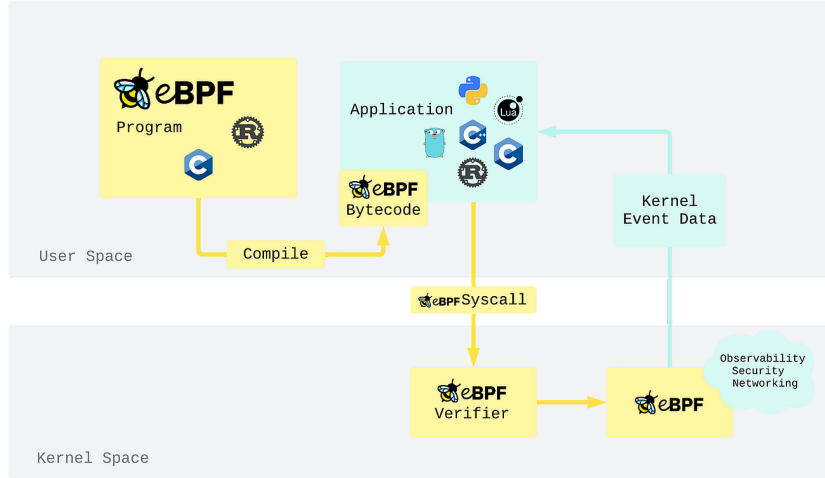


Figure 2.1: High-level eBPF execution workflow.

JIT compilation takes place only after verification and does not change the

authorised program semantics. Whether it is available depends on the host architecture, kernel configuration and runtime JIT settings. The complete path from compilation to invocation is shown in Figure 2.1 [16].

2.2.2 Object Files, Loading and Attachment Lifecycle

The source code of an eBPF application is not passed directly to the kernel. Clang first compiles it into BPF instructions stored in an ELF object. The object groups programs into named sections and also carries map declarations, licence information, symbols, BTF metadata and relocation records. Section names help a loader infer the program type and expected attachment family, while BTF and relocation information describe references that must be adapted before the program can be accepted by the target kernel [53, 44, 41].

A user-space loader, commonly built on `libbpf`, opens this object and turns its declarations into kernel objects. It creates or reuses maps, resolves relocations and submits each enabled program through the BPF system-call interface, after which the kernel verifier examines every program. A valid object can still fail to load if the host lacks a required program type, helper or symbol, or if its verifier handles the program differently [23, 48, 21].

Loading and attachment are separate operations. A successfully loaded program exists as a kernel object referenced from user space, but it does not run until it is connected to a compatible hook. Depending on the program and hook type, this connection may be represented by a BPF link or by tracing-specific infrastructure such as a perf event. The attachment owns the relationship between the program and its trigger and can be removed independently from the program itself. This separation allows one object file to contain several programs, only a subset of which may be loaded and attached for a particular monitoring configuration [53, 48].

Programs, maps and links are configured and released through file descriptors. Closing the final reference normally allows the kernel object to be destroyed; pinning instead assigns it a persistent name in the BPF filesystem, so it can outlive the process that created it. Cleanup is therefore part of runtime ownership rather than a minor loader concern: stale attachments and pinned objects may continue to consume kernel resources [23, 48].

2.2.3 Hook Selection and Event Semantics

An eBPF program type defines the execution contract enforced by the kernel, including the expected context, available helpers, and permitted return semantics. The hook instead identifies the event or kernel execution point that triggers the program, while the attachment mechanism binds the loaded program to a compatible hook. These concepts are related, but they are not interchangeable [53, 22].

The selected observation point determines not only when an eBPF program runs, but also the meaning of the resulting event. A system-call entry hook captures the operation requested by a process and its original arguments. A later tracepoint, dynamic probe, or LSM-related hook may expose kernel-resolved objects or the security mediation stage of the operation. An exit or return-side observation can instead reveal whether the operation completed successfully [55, 47, 49].

These observations belong to different stages of one kernel execution path. No single hook necessarily exposes the original request, resolved kernel context and final result, so a monitor may need to correlate records from several sources through process identity and time.

Compatibility also affects hook selection. Tracepoints provide structured interfaces where the target kernel exports a suitable event. Dynamic probes allow more flexible placement but depend more heavily on symbols, signatures and calling conventions. The required security evidence must be balanced against the capabilities and stability of the deployment kernel [55, 47, 25].

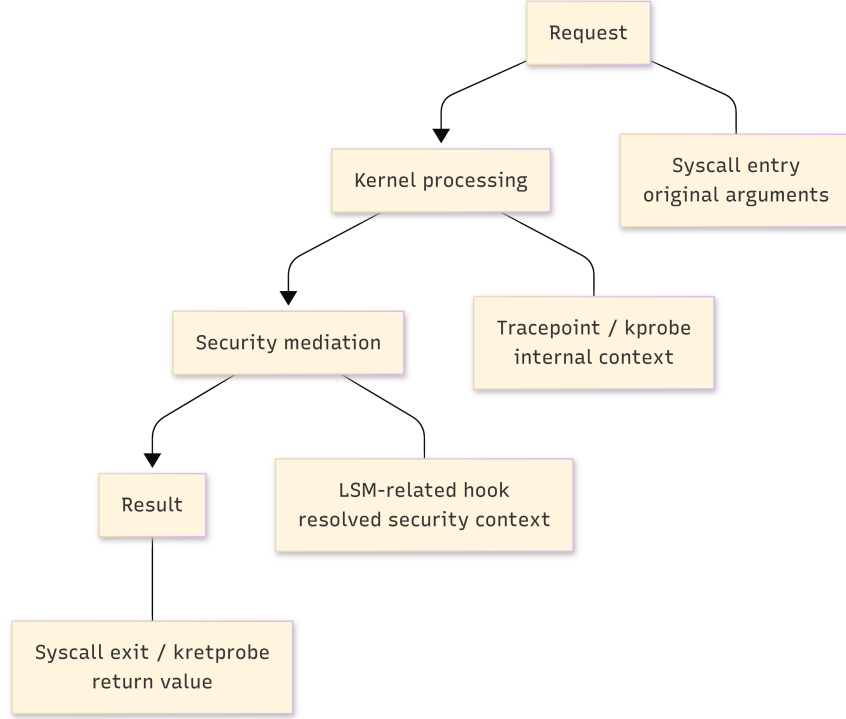


Figure 2.2: Complementary evidence exposed by different observation points along a kernel operation.

2.2.4 Maps, Helpers and Kernel-to-Userspace Transport

An eBPF invocation is constrained and usually short-lived, whereas monitoring often needs state to survive between invocations and eventually reach user space. Maps provide kernel-managed storage, helpers expose selected kernel services and event buffers carry records to the user-space runtime [42, 22, 52, 43].

Maps are typed kernel objects that allow eBPF programs to preserve state across invocations and exchange data with user space. Each map type defines its own access and concurrency semantics, while its structure and capacity are established at creation time. By moving persistent configuration and temporary state outside the constrained BPF stack, maps extend the information that a program can retain between executions. Accesses performed by eBPF code remain subject to verifier validation and to the rules

imposed by the selected map type [42, 23].

Map organisation reflects how the stored state is owned and accessed. Arrays offer fixed indexed storage for configuration or bounded workspaces. Hash maps use dynamic keys and can retain entry arguments until a later hook completes an operation. Per-CPU maps give each processor an independent value, reducing contention on frequently updated state or counters, although user space must combine those values to obtain a system-wide result. A shared map instead requires explicit consideration of concurrent access and update atomicity [42].

The same maps also carry configuration in the opposite direction. User space can populate selection or lookup data before monitoring starts and update supported values while programs remain loaded. Kernel programs can consult this state without encoding every operational choice in their instructions, leaving interpretation and configuration under user-space control [42, 23].

Helpers expose selected kernel services without granting programs unrestricted access to kernel internals. The verifier checks their arguments and the data passed to them. Availability depends on both program type and target-kernel capabilities; a helper documented for eBPF in general is not necessarily valid in every context. Support must therefore be checked on the deployed kernel rather than inferred from current upstream documentation [22, 21].

Event transport carries records produced during eBPF execution to user space, where they can be decoded and processed without extending the work performed inside the kernel. The BPF ring buffer provides a shared multi-producer, single-consumer channel designed to use buffer capacity across CPUs and preserve the order in which records are reserved. Perf-event transport follows a different model: a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` directs records to per-CPU perf ring buffers, which are consumed independently and can report lost events.

Both mechanisms have finite capacity. If kernel-side producers outpace the consumer, submission may fail or records may be lost. Transport is therefore part of the monitor’s correctness model rather than a transparent channel [43, 52, 22].

Mechanism	Buffer organisation	Main implication
BPF ring buffer	Shared multi-producer, single-consumer buffer	Shares capacity across CPUs and preserves reservation order between producers.
Perf-event transport	Per-CPU buffers referenced through a perf-event array	Buffers are consumed independently and provide explicit lost-event notifications.

Table 2.2: Comparison of eBPF event-transport mechanisms [43, 52].

In either case, correct decoding depends on a record format shared by kernel and user space [16].

2.2.5 BTF and CO-RE Portability

Kernel data structures can change layout between versions and distribution builds. An eBPF program compiled against one set of headers may therefore use field offsets or types that do not match the target. BTF and CO-RE preserve the meaning of compatible accesses by describing types and relocating them for the deployed kernel.

BTF is a compact representation of types and related metadata understood by the kernel and BPF tooling. It can describe C types and strings as well as function, line and relocation information. When the kernel exports BTF, loaders and debugging tools need not rely solely on headers copied from one build [44].

CO-RE uses BTF information to adapt an eBPF program to the kernel on which it is loaded. During compilation, Clang stores relocation records that describe the kernel types and fields accessed by the program. At load time, `libbpf` reads the BTF information of the running kernel and updates the relevant eBPF instructions so that they use the actual layout of those structures. The same object can consequently remain usable when a compatible kernel places a field at a different offset, without being recompiled on the monitored system [44, 41, 48].

CO-RE only adapts compatible differences in type and data-structure layout. It cannot make a missing hook, helper, program type or attachment mechanism available, nor can it ensure that the target verifier will accept a program. These capabilities must still be checked on the deployed kernel,

especially on enterprise distributions whose backports may not be apparent from the upstream version number [40, 25].

2.3 Related eBPF Runtime Security Tools

Runtime security tools differ in where they select events, interpret evidence and act on a match. The comparison here uses Tracee, Falco and Tetragon to mark three relevant positions in that design space: an event-oriented pipeline, a declarative alerting engine and a hook-centred system with optional response [19].

2.3.1 Tracee

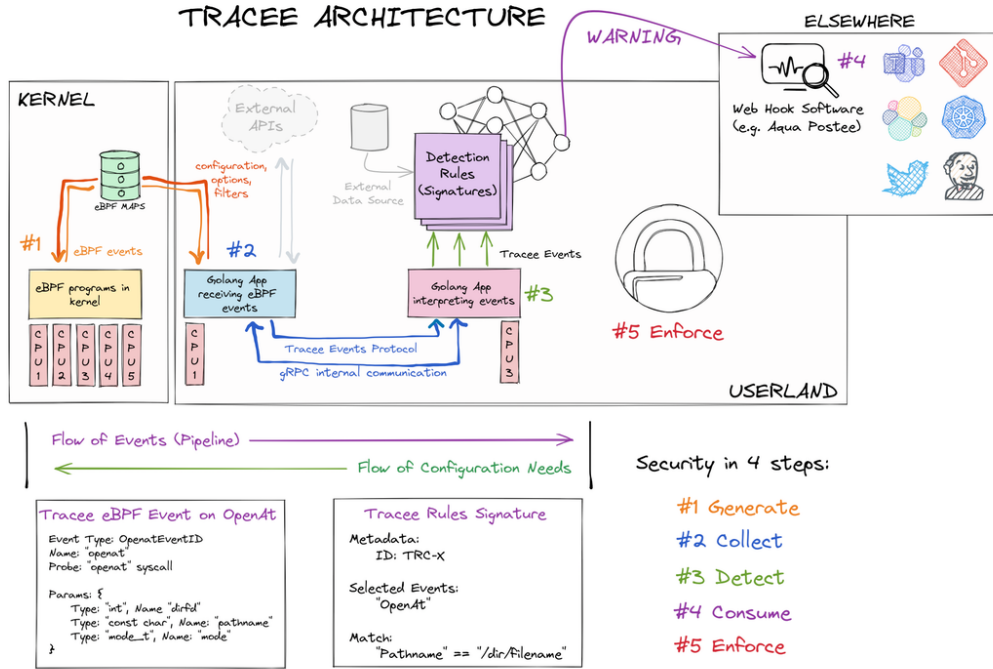


Figure 2.3: Tracee high-level architecture [3].

Tracee is the closest architectural reference for the proposed system because it separates eBPF collection from event processing, policy selection, detection and output. In the pipeline shown in Figure 2.3, kernel and networking hooks produce observations that become events before higher-level analysis and delivery [7, 3, 19].

The figure uses the signature-oriented terminology of the release in which it was published. Current documentation keeps the same broad pipeline but favours detectors for new threat detections and derived events. A common event abstraction represents both hook observations and detector results, keeping policies, detectors and output independent of physical attachment details [9, 4, 6].

Tracee policies combine workload scope with rules that select events and may constrain their data. Policy rules can also select detectors through threat metadata. Detectors consume declared inputs, may retain state and produce derived events with provenance. Monitoring scope and security interpretation are thus separate, without forming a general-purpose temporal language [11, 12, 5].

Output remains independent from event production and detection, supporting human-readable and structured destinations together with additional process and container context. Tracee also provides routing and forensic capabilities that extend beyond the prototype developed in this thesis [10, 8].

Tracee faces portability and trust boundaries as well. BTF, symbols, backports and individual hooks must be verified on enterprise kernels even when the general kernel line is supported. It also assumes that the kernel is trustworthy when monitoring starts; a kernel-level adversary may interfere with observation [1, 13].

The proposed system adopts this separation between collection, policy selection, detection and output, but not Tracee’s breadth. Its scope is a selected process and host-security event domain, using deterministic point conditions and bounded local sequences. The comparison establishes prior art and architectural influence, not functional equivalence.

2.3.2 Other Relevant eBPF Security Tools

Falco is a runtime security engine organised around rules evaluated over event sources. Kernel system-call activity can be captured through its driver

architecture, while plugins provide additional sources. Falco rules are declarative YAML objects containing a condition, output description, priority, and optional reusable lists, macros, tags, and exceptions. A matching rule produces an alert that can be routed to configured outputs [30, 33, 31].

Falco shows how readable rules, reusable expressions and explicit exceptions can support an alert-oriented interface. It also maintains process and container context for event enrichment. Its documented conditions evaluate an incoming event, however, and separate sources are processed independently; the core rule language is not presented as a temporal sequence language across heterogeneous sources [29, 32].

Tetragon combines eBPF-based observability with optional runtime enforcement. Its tracing policies specify kernel hooks, captured arguments or return values, and selectors that can match process, namespace, capability, workload, and argument attributes. Supported actions can include emitting an event, delivering a signal, or overriding a return value where the hook and kernel support that operation [61, 62, 58].

Tetragon places more selection and potential response near the hook than a design centred on user-space detectors. It preserves process-lifecycle context and emits structured events through its own interfaces, but its documented tracing-policy model is principally hook-centred rather than a general language for arbitrary temporal sequences [59, 61]. Its current installation requirements also illustrate the dependence of advanced eBPF functionality on kernel capabilities and BTF availability [60].

2.3.3 Comparative Summary

Table 2.3 places the three systems side by side. Tracee is the closest reference for the prototype’s event-oriented pipeline; Falco and Tetragon mark the neighbouring rule-centred and enforcement-capable designs. The comparison is architectural, since the tools differ substantially in scope and are not interchangeable.

Tool	Core approach	Role in the comparison
Tracee	Event-oriented pipeline with separate policy, detection, and output stages.	Closest architectural reference for the proposed system. [11, 6]
Falco	Declarative rules evaluated over incoming events to generate alerts.	Reference for a mature rule- and alert-centred model. [33, 32]
Tetragon	Hook-centred policies with in-kernel selectors and optional actions.	Reference for kernel-side filtering and runtime enforcement. [61, 58]

Table 2.3: Architectural summary of the runtime security tools considered in this chapter.

2.4 Positioning of the Proposed Work

The proposed work covers a bounded part of Linux runtime security: process and host-security monitoring on the enterprise environment specified in Chapter 3. Its design assumes only the capabilities verified on that target.

The prototype does not claim these mechanisms as novel in isolation. Tracee establishes the event-oriented separation of collection, policy selection, detection and output; Falco provides a mature example of declarative event rules and alerts; and Tetragon illustrates a more kernel-centred design with enforcement support [11, 6, 33, 61].

This thesis integrates those established ideas in a compact prototype tailored to the selected enterprise kernel. The work concentrates on compatibility, complementary observation points and the contract that turns kernel records into typed user-space events. Chapter 3 develops the corresponding system design.

Chapter 3

Requirements and System Design

Chapter 2 introduced the kernel mechanisms on which the monitoring system is built. This chapter turns those foundations into concrete requirements and an overall system design. Particular attention is given to the constraints of the target environment and to the boundary between kernel-space collection and user-space processing.

Code that runs in the kernel is kept small and bounded. Interpretation takes place in user space, where it can change without adding verifier-sensitive logic to every eBPF program. A shared event contract links the two sides. Policies use the resulting events to define the monitored scope, and detectors look for configured conditions in individual observations or short ordered sequences. This chapter describes those choices as architecture; Chapters 4 and 5 cover their kernel- and user-space implementations, and Chapter 6 evaluates the resulting system.

3.1 Design Goals and Constraints

The system targets security-relevant host activity, with an emphasis on process execution and operations that can alter the host security state. It is not intended to reproduce every kernel action. Operators must be able to choose the required event domain, while all selected observations retain a consistent typed representation. Detection logic then works on this representation

rather than being embedded in the collection programs.

Runtime telemetry is continuous, so unnecessary collection and unbounded detection state would steadily consume resources. Losses and processing failures must also be reported explicitly, since silence alone cannot show that no relevant operation took place. Compatibility constraints are defined for the target environment below.

3.1.1 Target Environment and Compatibility Assumptions

The target environment is Rocky Linux 8.10 on x86-64, running the exact kernel build `4.18.0-553.137.1.el8_10.x86_64`. This target belongs to a long-lived enterprise kernel line in which selected fixes and features are backported while the reported upstream base version remains stable. The nominal version therefore cannot be used alone to infer the effective eBPF capabilities of the host; the functionality required by the monitoring system must be confirmed on the deployed kernel [25].

The architecture assumes that the monitored host exposes usable kernel BTF information and grants the privileges required to load and attach the selected programs. As discussed in Chapter 2, BTF and CO-RE can adapt compatible differences in kernel data-structure layout, but they cannot provide missing kernel functionality, bypass permissions, or guarantee verifier acceptance [44, 41, 48, 21].

Portability is therefore limited to compatible systems and not assumed for all Linux distributions and kernel configurations. Dynamic probes also depend on available symbols and compatible function signatures. The runtime must detect loading or attachment failures before monitoring starts, otherwise the active event coverage could differ silently from the requested one.

3.1.2 Functional Requirements

A functional requirement states what the system must allow a user to do or what behaviour it must provide. For this monitor, these requirements cover event selection, collection, analysis and reporting. Table 3.1 lists the required behaviours; the paragraphs that follow explain why they are needed.

ID	Functional requirement
FR-01	Let the operator choose which supported types of host activity to monitor.
FR-02	Collect observations and make their fields available in a common format for analysis.
FR-03	Let the operator filter observations and choose which security checks to enable through configuration files.
FR-04	Check individual observations and sequences of related operations against configured conditions.
FR-05	Generate alerts that identify the matching security check and include the observations that triggered it.
FR-06	Provide readable and machine-readable output for observations and alerts, with separate operational diagnostics.
FR-07	Check configuration before starting, report failures and release resources when monitoring stops.

Table 3.1: Functional requirements of the monitoring system.

FR-01 allows monitoring to follow the operator’s needs. The operator can select all or part of the software’s supported event catalogue. For example, an investigation focused on file access may require only the relevant file events. Unsupported event names must be reported as configuration errors.

FR-02 makes observations from different sources usable by the same analysis components. Each event must identify the observed operation and expose its available context such as the process involved, the accessed resource or the returned result. The system must preserve the meaning of these fields.

FR-03 lets operators narrow monitoring to a particular scenario without changing the software. Configuration files called policies can select or exclude activity using event names, process names, user identifiers and operation details. For example, a policy may retain file-access events involving a particular path. Policies can also restrict the active security checks, called detectors, by their identifiers or descriptive attributes, such as severity and tags.

FR-04 allows the software to recognise activity that requires different amounts of evidence. A check may concern one operation (access to a specified file) or a sequence (execution of a program followed by access to that file). Sequence checks must define how the observations are related and the time allowed between them.

FR-05 allows an analyst to inspect why an alert was raised. An alert must name the detector that matched and retain the relevant observations in their sequence order where applicable. It must also report the configured severity and any supplied security classification metadata.

FR-06 supports both direct inspection and further processing of results. Text output allows terminal use, while JSON allows other software to read the observations and alerts. Operational messages, such as collection or processing errors, must be distinguishable from the monitored activity.

FR-07 supports controlled operation of the monitor. Invalid configuration must be reported before collection begins, and failures that prevent startup must be visible to the operator. When monitoring is stopped, or startup fails after resources have been acquired, those resources must be released.

3.1.3 Non-Functional and Operational Requirements

Functional requirements describe what the software must do, such as collect events or generate alerts. Non-functional requirements describe the conditions under which those functions must operate, such as the systems the software must support and the resources it may consume. For this monitor, they define the target Linux environment and the limits on collection, stored detection data and CPU usage. Table 3.2 lists these requirements.

ID	Non-functional requirement
NFR-01	Support Rocky Linux 8.10 on the x86-64 kernel specified in Section 3.1.1.
NFR-02	Limit kernel-side data collection through fixed buffer capacities and traversal limits. Collection programs must pass the target kernel's eBPF verifier.
NFR-03	Keep mean CPU consumption below 5% of one core in the UID-filtered evaluation profile, measured for the monitor's container cgroup.

Table 3.2: Compatibility and resource requirements of the monitor.

NFR-01 establishes the environment in which the monitor must work. Kernel versions differ in the information and monitoring facilities they provide. Specifying the target makes compatibility testable: the required programs must load and attach successfully on that system.

NFR-02 controls the work performed whenever a collection program runs. The eBPF verifier checks these programs before the kernel accepts them, including their memory accesses and execution paths. The collector must therefore impose explicit limits on the data it reads. For example, the design limits path reconstruction to 20 components and the argument buffer to 32,000 bytes. These are separate limits for different operations; Chapter 4 explains their implementation. They keep collection predictable when an operation exposes large amounts of data.

NFR-03 expresses the project’s resource target for the controlled experiment. Monitoring shares the host with the applications being observed, so its CPU demand must be measured under a defined workload. The threshold applies to the profile using an exact UID filter and to CPU time charged to the monitor’s container cgroup. Chapter 6 specifies the workload, configuration and measurement procedure, and compares this profile with filtering disabled.

3.2 System Architecture

The architecture has a control path and a data path. The control path validates configuration and rule definitions, selects events and detectors, and maps each logical event to the kernel programs needed to observe it. The data path begins when one of those programs runs. It carries the captured record through transport and user-space processing, producing an event or, after a detector match, an alert. Operational diagnostics report problems in either path without becoming monitoring data themselves.

3.2.1 Kernel-Space and User-Space Responsibilities

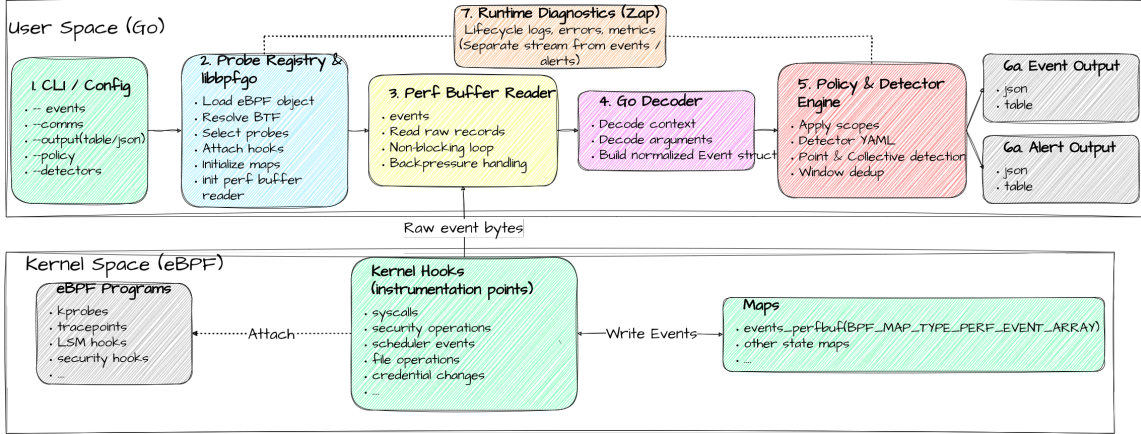


Figure 3.1: Kernel-space and user-space responsibilities.

Figure 3.1 shows the responsibilities on either side of the kernel–user-space boundary and the interactions between them.

In kernel space, selected eBPF programs collect process context and operation-specific evidence, construct bounded records and submit them through the perf-event channel. Some events require short-lived state to combine an entry observation with its return. An optional exact effective-UID filter can reject a record before submission. Policy evaluation and security classification are not performed at this stage.

User space manages configuration and the parts expected to change more often. The Go runtime determines the event scope, translates logical events into a probe set, loads and attaches programs through `libbpfgo`, decodes records and runs policies and detectors [2]. Events, alerts and operational diagnostics use separate output paths because they represent different kinds of information.

This division keeps kernel work constrained and lets the interpretation layer change independently. It also exposes an important limitation: user space cannot recover a record rejected in kernel space or lost before decoding.

3.2.2 End-to-End Event Lifecycle

Before collection begins, the validated configuration, policies and detector definitions establish the logical event scope. The probe registry expands it into public producers and any required support probes; unrelated programs stay disabled. During monitoring, a hook produces a binary record, which is transported and decoded against the event registry. User-space policy constraints then admit or reject the event. Admitted events can be presented and evaluated by detectors. A successful point or sequence match produces an alert containing the source evidence and available MITRE ATT&CK metadata. Operational diagnostics remain outside this data path: they report runtime conditions without becoming monitored events or detector evidence. The event lifecycle also does not establish a total order across observations. Perf-event transport uses per-CPU buffers, so collective detectors use the order in which records reach the user-space engine. Section 3.4.2 discusses this ordering constraint together with transport loss.

3.3 Kernel-Space Collection Design

Kernel instrumentation must provide evidence that is meaningful for the logical event and available on the target kernel. No single attachment mechanism meets both needs in every case, so the design combines several kinds of observation point behind one stable event abstraction.

3.3.1 Hook Selection and Logical Events

Hooks are selected according to the information required by each event and their availability on the target kernel. The choice also considers how often the attached collection program will run. A hook reached by many unrelated operations requires repeated checks to discard activity outside the selected scope. Where possible, the design therefore uses hooks specific to the operation being monitored.

Dedicated tracepoints are preferred when they expose the required information. Dynamic probes provide access to kernel functions when suitable tracepoints are unavailable or insufficient. Multiple hooks may be needed to observe both the request and the outcome of an operation.

Table 3.3 distinguishes the architectural concepts used to describe instru-

mentation and event production.

Concept	Architectural meaning
Physical hook	A specific point in kernel execution, such as a tracepoint or kernel function, at which an eBPF program can be attached and invoked.
Producer probe	An eBPF program that directly captures an observation and submits a record for a public logical event. A logical event may be produced by one or more producer probes.
Support probe	An internal eBPF program that does not produce a public event of its own. Instead, it stores, derives, or supplies information required by a producer probe to construct another event correctly.
Logical event	The stable user-space event type exposed by the monitoring system, identified by a public name and decoding schema. It hides the number and type of physical hooks and probes used to construct the observation.

Table 3.3: Distinction between physical hooks, producer and support probes, and logical events.

The operator selects logical events without having to identify the individual kernel programs required to collect them. The system maps each selection to its producer probes and support dependencies. This allows the collection mechanism to change while preserving the public event name and schema.

3.3.2 Early Event Selection and Kernel-Side Filtering

Early selection operates on logical events, so operators do not need to name individual eBPF programs. For each requested event, the probe registry finds its producers and any support probes they require.

This dependency set determines which registered programs enter the loading lifecycle. Programs outside it are disabled before kernel verification and are not attached. The active observation surface is reduced without dropping the internal programs needed to construct a selected event.

Selective loading covers programs represented in the probe registry, not every section that could exist in the compiled object. Correctness depends on a complete description of producers and dependencies. A missing dependency

can leave a selected event incomplete; exposing a support probe incorrectly can turn an internal mechanism into an apparent public event.

An optional exact UID guard provides a second reduction after a program has been invoked. It compares the task's effective UID with one configured value and rejects a mismatch before full context enrichment, serialisation and transport. This avoids work for observations known to be outside that narrow scope.

The guard deliberately supports only one equality condition. Other cheap predicates, such as process or local cgroup identity, could be added if their inputs are reliable and the reduction is measurable. Richer conditions stay in user space, where the complete normalised event is available.

3.4 Event Contract and Transport Design

The event contract is the agreement between the kernel programs that collect observations and the user-space runtime that reads them. It defines how each event is identified, which fields it contains and how their values are represented. A common format allows observations from different hooks to reach the same analysis components.

3.4.1 Stable Binary Event Contract

Each record contains a common context, an argument count and a sequence of event-specific arguments. The context records shared information, including the event identifier, timestamp and process identity. The arguments describe the operation observed, such as the file accessed or the credentials changed. Separating these parts allows events to carry different details while sharing the information needed to identify their source.

Each argument begins with an index followed by its encoded value. The event registry associates each event ID with a schema containing the argument names and types. The decoder uses the event ID to select the schema, then uses each argument index to find the corresponding field. Field names and type descriptions are kept in the registry, reducing the metadata repeated in transmitted records.

Figure 3.2 illustrates this design through a `security_file_open` event. The example shows three selected arguments with illustrative values; other ar-

guments are omitted.

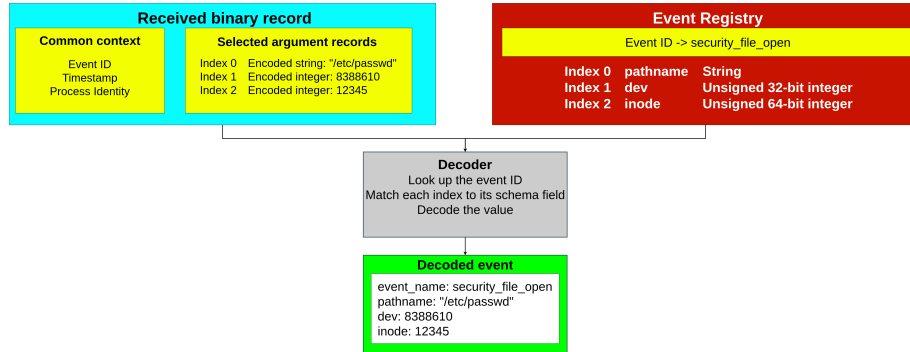


Figure 3.2: Interpretation of selected `security_file_open` arguments using the event registry.

The event ID selects the `security_file_open` schema. Index 0 identifies `pathname`, whose value is decoded as a string. Indices 2 and 3 identify `dev` and `inode`, decoded as unsigned 32-bit and 64-bit integers, respectively. The resulting event contains these named fields together with the common context.

Indices also allow individual arguments to be omitted without changing the meaning of the remaining fields. Whether a producer continues after a collection failure depends on the missing information. For example, `security_file_open` stops without submitting a record if it cannot obtain the resolved path. When a record is submitted, its argument count states how many arguments were written.

The producer and decoder must agree on identifiers, indices, field widths, byte order and variable-length encodings. These definitions are maintained separately in C and Go. Registry tests check identifiers and names, while decoder tests exercise selected encoded records. Changes to the format therefore require review of both the producer and the receiving schema.

The decoder reports unknown event identifiers, indices outside the selected schema and payloads that fail their type-specific checks. Chapter 4 describes record construction, and Chapter 5 covers decoding.

3.4.2 Perf-Buffer Transport and Failure Boundaries

Records are transferred through per-CPU perf buffers using a `BPF_MAP_TYPE_PERF_EVENT_ARRAY` map. This transport was selected for its availability on the target kernel. A collection program submits a record to the buffer associated with its current CPU, and the user-space runtime reads it for processing [52].

The buffers hold records while collection and processing proceed at different rates. Their capacity is finite: if records arrive faster than the runtime reads them, a buffer can fill and subsequent records may be lost.

The runtime addresses temporary increases in activity by separating reading from processing. A dedicated reader retrieves records and places them in a queue while the processing loop decodes and analyses earlier events. This allows reading to continue during short processing delays, provided the queue still has space.

Selective collection also reduces the incoming workload. Programs unrelated to the selected events remain disabled, and the optional UID filter rejects observations outside its configured scope before transport. Each submission includes only the populated part of the record, reducing buffer use.

If processing remains slower than collection, the queue eventually fills and blocks the reader. The perf buffers may then fill as well. The runtime reports loss notifications from the reader as warning messages. These contain the number of reported losses, but no information from which the missing records can be recovered. Decoding failures are logged separately to distinguish them from transport losses.

Missing observations may prevent a detector from completing a sequence. Collection diagnostics therefore help interpret an interval without alerts. Chapter 6 reports whether perf-loss warnings appeared in the retained experimental logs.

3.5 User-Space Runtime Design

The Go runtime connects kernel collection to analysis. Lifecycle management, record interpretation and presentation have separate responsibilities, giving each resource and failure a clear owner. Their implementation is described in Chapter 5.

3.5.1 Loading, Attachment and Runtime Lifecycle

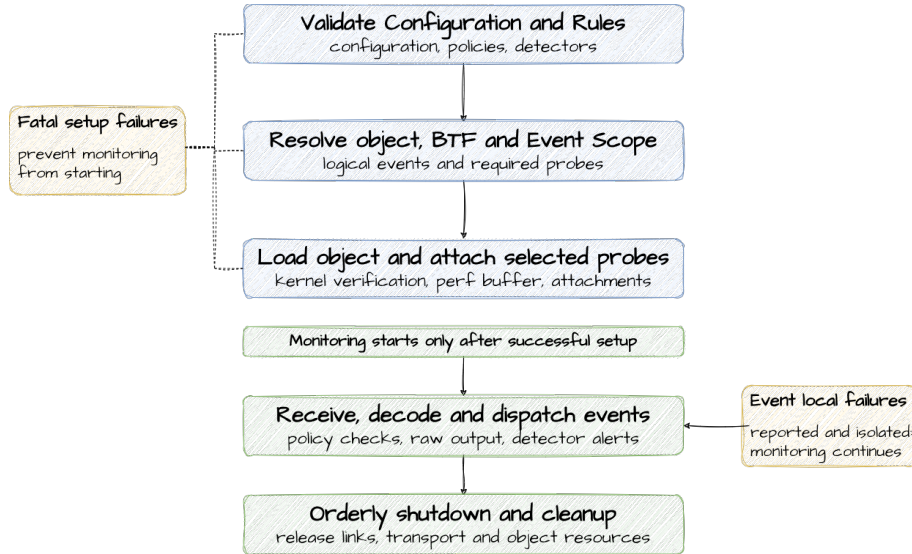


Figure 3.3: Simplified user-space runtime lifecycle.

The runtime moves through validation, setup, active monitoring and cleanup, as shown in Figure 3.3. Configuration or setup failures prevent the session from starting, whereas failures confined to one record are reported without stopping later processing. The runtime retains ownership of acquired resources and releases them after both complete and partial initialisation.

3.5.2 Decoding, Normalisation and Event Registration

The decoder is the boundary between transported bytes and the typed events shared by policies, detectors and output. The event registry defines their public identifiers and argument schemas, while the separate probe registry maps them to producer and support programs. Records without a valid schema are rejected before reaching downstream components. Chapter 5 describes the decoder and the consistency checks between these registries.

3.5.3 Event Output, Alert Output and Operational Logging

The architecture distinguishes admitted events, detector alerts and operational diagnostics. Suppressing ordinary event output does not disable detector evaluation, and runtime failures are never represented as monitored activity. Chapter 5 covers their formats and error handling.

3.6 Policy and Detection Architecture

Policies select the activity relevant to a monitoring task and the detectors to enable. Detectors examine the selected observations and generate alerts when their conditions match. Before monitoring begins, the runtime prepares a plan containing the events to collect, the filters to apply and the detectors to run. These choices remain fixed during the monitoring session.

3.6.1 Policy-Based Event Scope

Policies can filter observations by event name, process command, UID and argument values. They can also select detectors by identifier or metadata, such as severity and tags.

Table 3.4 shows an illustrative plan for investigating execution and file access by processes running under UID 1000. The example assumes one active policy and an available detector definition named `file-access-check` that examines file-opening events.

Plan element	Example
Operator event selection	<code>sched_process_exec</code> , <code>security_file_open</code> , <code>chmod</code>
Policy event selection	<code>sched_process_exec</code> , <code>security_file_open</code>
Resulting collection	<code>sched_process_exec</code> , <code>security_file_open</code>
Event filter	UID equals 1000
Active detector	<code>file-access-check</code>

Table 3.4: Illustrative monitoring plan for execution and file access.

The operator initially selects three event types. The policy retains two, so the programs used only for `chmod` are left disabled. The UID condition is

checked after decoding: events from other UIDs reach neither event output nor detectors. This condition does not automatically enable the separate kernel UID filter.

Before monitoring starts, the system compares the events selected by the operator with those listed in the policies. When all rules that admit activity list the events they need, collection is limited to the event types shared by the two selections. If a rule allows any event type, the operator's selection is kept. Conditions on process names, UIDs and argument values are then checked after the events have been collected and decoded.

All active policies contribute to one shared plan. An event must match an allowing policy, and a matching suppression policy overrides that decision. Without policies, events pass this gate. Selecting detectors determines which checks run; it does not independently enable additional kernel events.

3.6.2 Point and Collective Detectors

A point detector evaluates one event against configured conditions. For example, `file-access-check` could generate an alert when a file-opening observation identifies a specified sensitive path.

A collective detector evaluates an ordered sequence of related events. For example, a check may require a credential change followed by program execution by the same process. Events are considered in their arrival order. A configured time window limits how long the runtime waits for completion, starting when it stores the first matching observation.

Both forms use explicit, deterministic conditions and consume the same decoded event model. Chapter 5 explains how their definitions are loaded and evaluated.

3.6.3 Bounded Local Correlation

Sequence detection requires a way to identify observations that belong together. Several processes can share a UID, so that value alone cannot establish that two events came from the same process. Process correlation uses the host PID and process start time, distinguishing separate lifetimes that reuse the same PID.

Other supported relationships use the immediate parent process, a file's device and inode numbers, or a local cgroup identifier. Several dimensions

can be combined when all must agree.

Incomplete sequences are subject to the time and capacity limits stated in Section 3.1.3. Unsupported correlation strategies are rejected during configuration. An event that cannot provide a valid correlation key does not advance a sequence. Chapter 5 describes how this state is stored and removed.

3.6.4 Alert Evidence and MITRE ATT&CK Metadata

An alert identifies the detector, reports its configured severity and preserves the observations that satisfied its conditions. The analyst can inspect this evidence to understand why the alert was raised.

Detector definitions may include MITRE ATT&CK labels describing the attack tactics and techniques they are intended to identify [26]. Policies can use these labels to choose which detectors to enable.

3.7 Design Decisions and Scope Boundaries

The design favours reliable observation on the target host over the broadest possible event catalogue. Selective loading and explicit probe dependencies limit collection to the event surface supported by that host.

Early filtering is used only where it stays small and verifier compatible. Policies and detector semantics remain in Go, where they can change without recompiling the collection layer. The shared event contract supports this division, and perf-event transport reflects the facilities available on the reference kernel.

Detection state is similarly constrained. Point detectors and bounded local sequences add interpretation to raw telemetry without maintaining persistent graphs, learning statistical models or retaining unbounded correlation state. The prototype observes and detects; it does not enforce decisions or provide a general-purpose event-processing service. Chapter 6 evaluates node-local event visibility, the configured collective detector and monitor-group resource consumption under the selected profiles.

Alert aggregation across hosts and the addition of external deployment context are left to a wider platform and are neither implemented nor evaluated here. The system considered in this thesis ends at node-local, evidence-

preserving detection.

Chapter 4

Kernel-Space Event Collection Implementation

This chapter describes the kernel-space implementation of the design presented in Chapter 3. It begins with the build artefacts and common event-construction code, then examines the implemented hook families, binary record format, perf-buffer transport, early UID admission and compatibility work for the target Rocky Linux kernel. Chapter 5 takes over at the user-space boundary; Chapter 6 evaluates the completed system.

4.1 Build Toolchain and Project Organisation

The build has three ordered stages. The libbpf source under `3rdparty/libbpfgo/libbpf/src` is first compiled into the local static library `dist/libbpf/obj/libbpf.a`, with its headers and package metadata. Clang then compiles the eBPF source for x86-64 into `dist/project.bpf.o`. Finally, CGO links the Go application against the local libbpf archive and libbpfgo bindings to produce `dist/project`. The Makefile enforces this order and avoids silently using system copies of the kernel-facing dependencies.

The Go executable embeds the eBPF object, so `dist/project.\gls{bpf}.o` must exist before the Go package is compiled. The normal runtime path does not require a second object file beside the executable, although an external object can be supplied for development and compatibility testing. Embedding does not remove host dependencies: loading still requires suit-

able kernel BTF, privileges and the libraries used by CGO.

4.2 eBPF Program Implementation

The eBPF object contains programs for several hook types. Most records follow one of two construction patterns. A direct producer creates and submits an event during one invocation. A paired producer saves request arguments at entry and completes the event when a return hook supplies the result.

Physical hooks and public events are not in one-to-one correspondence. Several programs may produce one logical schema, while support programs can be attached without emitting public events. Section 5.2 explains how the loader turns these relationships into an attachment plan.

4.2.1 Shared Event-Construction Framework

Both patterns build records in a reusable per-CPU workspace instead of placing the full event structure on the limited eBPF stack. Other per-CPU maps hold temporary path and kernel-object data. Task and process maps retain context that can be reused across invocations.

The common initialiser clears the argument buffer and records basic identity and timing data. The hook appends its own fields, after which the submission helper refreshes the full task context and sends the populated record through the interface described in Section 4.3.

Listing 4.1 shows the main stages shared by these helpers. Details related to process-map initialisation and optional stack capture are omitted because they do not change the event-construction process.

```
1  if (!p->event)
2      p->event = bpf_map_lookup_elem(&event_data_map, &zero);
3  if (!p->event)
4      return 0;
5
6  p->config = bpf_map_lookup_elem(&config_map, &zero);
7  if (!p->config)
8      return 0;
9
10 reset_event_args_buf(p->event);
11 p->event->task = (struct task_struct *) bpf_get_current_task();
```

```
12
13     u64 id = bpf_get_current_pid_tgid();
14     p->event->context.task.host_tid = id;
15     p->event->context.task.host_pid = id >> 32;
16     p->event->context.eventid = event_id;
17
18     init_task_context(&p->event->context.task,
19                     p->event->task, p->config->options);
20
21     u32 size = sizeof(event_context_t) + sizeof(u8) +
22               p->event->args_buf.offset;
23     long ret = bpf_perf_event_output(
24               p->ctx, &events, BPF_F_CURRENT_CPU, p->event, size);
25
26     update_event_stats(p->event->context.eventid, ret);
27     return ret;
28
```

Listing 4.1: Shared event construction and submission path.

Submission copies the populated prefix into the perf-event channel, after which the per-CPU workspace can be reused. Later writes cannot alter the submitted record. If the workspace, configuration or task context is unavailable, the invocation ends without producing an event.

The full helper also checks the record size and can update delivery statistics. Listing 4.1 concentrates on the transport boundary; Section 4.3 covers the exact bounds.

Paired producers span two invocations. Entry arguments are stored in a map and retrieved when the return program adds the result. The key combines the logical event with the issuing host thread, preventing different operations on that thread from sharing the same slot.

```
1     statfunc u64 get_args_id(u32 event_id)
2     {
3         return ((u64) event_id << 32) |
4                (u32) bpf_get_current_pid_tgid();
5     }
6
7     statfunc int save_args(args_t *args, u32 event_id)
8     {
9         u64 key = get_args_id(event_id);
10        bpf_map_update_elem(&args_map, &key, args, BPF_ANY);
11        return 0;
12    }
13
```

```
14 statfunc int load_args(args_t *args, u32 event_id)
15 {
16     u64 key = get_args_id(event_id);
17     args_t *saved = bpf_map_lookup_elem(&args_map, &key);
18     if (!saved)
19         return -1;
20
21     /* Fixed argument-slot copies are omitted. */
22     return 0;
23 }
```

Listing 4.2: Saving and recovering arguments for paired producers.

The seven argument slots are copied explicitly so that each access remains statically bounded for the verifier. If lookup fails, the return program cannot reconstruct the request and emits no record.

The return program deletes the entry after copying it and then appends the result. Entry updates use `BPF_ANY`, so a later request can replace state left behind by a missed return. This map is temporary construction state, not a process history.

The hook families below reuse these patterns, but the meaning of a record still depends on its observation point. Syscall entry captures a request. A security hook exposes an operation under mediation, usually without the final decision. A committed-state hook sees installed state, and a lifecycle tracepoint reports a completed transition. Exit or return probes add the result visible at that boundary.

4.2.2 Process Lifecycle and Executable Transitions

Process creation is visible both as a syscall and as a scheduler transition. The syscall records the requested operation and returned result; scheduler tracepoints observe tasks after creation, rename or termination. Both views are kept because they answer different questions.

4.2.2.1 Process Creation and Lifecycle

The `fork` and `vfork` events provide the simplest syscall-level view of process creation. Their schemas contain only the return value, so their entry probes do not retain any arguments. A positive result identifies the child returned to the caller, while a negative result records the reported error.

The `clone` event captures the more configurable creation interface. Its entry probe retains the clone flags, stack address, parent and child thread-ID pointers and thread-local-storage value. These fields describe resource sharing and execution-context setup. The exit probe adds the returned child identifier or error.

The `sched_process_fork` tracepoint runs after the child task exists and receives both task structures. It records the host PID and TID of parent and child, together with the child's start time. The corresponding source excerpt is retained in Listing A.2 of Appendix A.

Because PIDs and TIDs can be reused, the start time lets user space distinguish separate task lifetimes with the same numeric identifier. The same combination is used as a process identity for local detector correlation.

Two direct tracepoints cover later lifecycle changes. `task_rename` records the previous and new command names when the task name changes. This operation is kept separate from executable replacement because a command-name change does not imply that a new program image was installed. The `sched_process_exit` event records the task's exit code and whether the complete thread group is terminating, distinguishing a single thread exit from the end of the process.

These lifecycle relations are emitted as bounded events; the kernel does not maintain a persistent process graph. Longer-lived reconstruction, where needed, belongs in user space.

4.2.2.2 Executable Transitions

Executable-transition monitoring retains the requested program as well as the outcome. Pathname and arguments are available at syscall entry, but that point cannot show whether the kernel installed the new image. Separate hooks cover the request, binary security processing and final result.

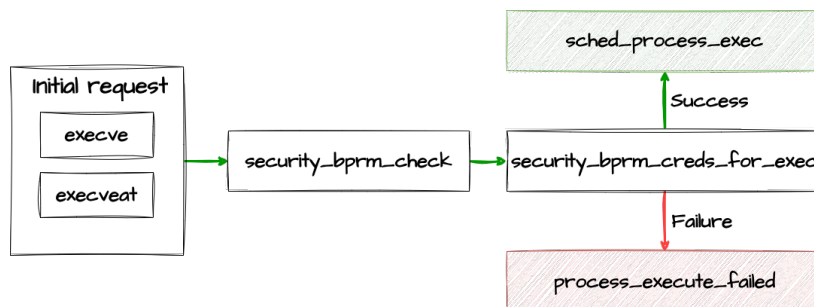


Figure 4.1: Events used to observe execution requests, kernel security processing and executable-transition outcomes.

Figure 4.1 shows coverage, not a guaranteed one-to-one sequence. Binary handlers and interpreters may trigger the security hooks more than once, and those hooks do not expose the final decision.

The `execve` and `execveat` events preserve the requested pathname and up to 38 argument strings while the user-space memory is still available. `execveat` also records the directory file descriptor and flags needed to interpret relative or descriptor-based execution. As entry events, neither contains a result.

`security_bprm_check` runs after the kernel has constructed `linux_binprm`. The event records the resolved pathname, device, inode, original filename, argument count and environment count, tying the request to a resolved kernel object.

`security_bprm_creds_for_exec` observes credential preparation. It captures the same binary identity and adds the file change time. Prepared credentials are not necessarily installed; committed transitions are covered in Section 4.2.3.1.

Two mutually exclusive events represent the outcome. `sched_process_exec` confirms installation of a new image and records its resolved filename. A failed `execve` or `execveat` produces `process_execute_failed`, retaining the operation variant, directory file descriptor, requested pathname, flags and negative return value.

The failure event uses paired entry and exit programs because the pathname and execution options are available before the syscall runs, while the error is available only when it returns. The return-side implementation is shown in Listing A.5 of Appendix A.

The return program removes saved state before testing the result. Success

only clears the temporary entry; a negative value is serialised with the original request. Separate entry and exit programs cover `execve` and `execveat`, but all four use one public schema.

Caller intent, resolved binary identity and outcome remain separate. An attempt is not mistaken for a completed transition, and user space can translate negative return values into readable errors.

4.2.3 Identity, Credentials, Process Control and Limits

4.2.3.1 Identity and Credentials

A set-ID syscall alone cannot describe every privilege transition. Credentials may also change during executable loading or through internal kernel paths, and a security hook sees a proposal before it becomes active. The implementation therefore records identity requests, mediation, committed credentials and capability checks at separate points.

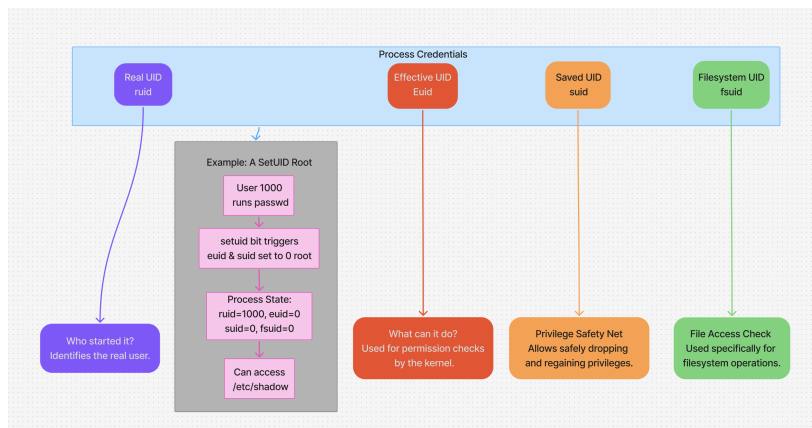


Figure 4.2: Real, effective, saved and filesystem UIDs for a process [15].

Linux associates each process with several user and group identifiers. The real ID represents the identity under which the process was started, while the effective ID is used for most access-control decisions. The saved ID allows a process to retain an effective identity that may later be restored, supporting temporary privilege reduction and recovery. Linux also maintains filesystem IDs, which are used specifically during filesystem permission checks. The

same distinction applies to user and group credentials [15].

A process can change its effective or filesystem privileges without changing its real identity. A saved identity may also allow dropped privileges to be restored. Monitoring only the real ID would miss these changes in effective authority [15].

Identity requests are captured through the `setuid`, `setgid`, `setreuid`, `setregid`, `setresuid`, `setresgid`, `setfsuid` and `setfsgid` events. Their entry programs retain one, two or three identifiers according to the syscall interface. This distinguishes single-ID, real/effective, real/effective/saved and filesystem-specific requests. Exit programs append the returned value.

Return semantics differ across this family. Most operations report a status, but `setfsuid` and `setfsgid` return the previous filesystem identity. User space therefore needs the originating event to interpret the value correctly.

The `security_task_fix_setuid` hook adds the mediation view. It records the old and proposed real, effective, saved and filesystem UIDs, together with a flag that distinguishes the set-ID operation category. The eBPF program compares the two credential proposals and suppresses the event when none of the monitored UID fields changes. Since the hook is observed before installation and its result is not captured, the record describes a transition under examination, not a confirmed change.

No public `security_task_fix_setgid` event is currently exposed. Group-ID requests remain visible through the set-group syscall events, while changes that become active are included in the broader `commit_creds` event.

The `commit_creds` hook runs when replacement credentials become active. It converts the installed and replacement credentials into fixed-width snapshots containing real, effective, saved and filesystem user and group IDs, the user-namespace ID, secure bits and the inheritable, permitted, effective, bounding and ambient capability sets.

Credential installation is frequent, and many invocations do not alter a field used by the monitor. The two snapshots are compared before submission. Listing 4.3 shows this change-only emission strategy.

```
1 slim_cred_t old_slim = {};  
2 slim_cred_t new_slim = {};  
3  
4 /* Identifiers, namespace, secure bits and capability  
5  * sets are copied into the two snapshots. */  
6
```

```
7  if (old_slim.uid == new_slim.uid &&
8      old_slim.gid == new_slim.gid &&
9      old_slim.euid == new_slim.euid &&
10     old_slim.egid == new_slim.egid &&
11     /* Remaining monitored fields are compared here. */
12     old_slim.cap_ambient == new_slim.cap_ambient)
13     return 0;
14
15  save_to_submit_buf(&p.event->args_buf,
16                   &old_slim, sizeof(slim_cred_t), 0);
17  save_to_submit_buf(&p.event->args_buf,
18                   &new_slim, sizeof(slim_cred_t), 1);
19
20  return events_perf_submit(&p);
```

Listing 4.3: Kernel-side suppression of unchanged credential snapshots (abridged).

Unchanged snapshots never enter the perf-event channel. When a relevant field differs, the event preserves both states, so analysis does not depend on finding an earlier credential record.

`cap_capable` records the capability number presented during a kernel capability check, including checks unrelated to a credential replacement. The event does not include the checked credentials, user namespace, option flags or final decision, so it cannot by itself establish a privilege change or a successful operation.

The resulting records distinguish an identity request, a mediated proposal, an installed credential state and a capability check instead of reducing all privilege-related activity to one event type.

4.2.3.2 Process Control and Limits

Process-control operations can inspect another task, change current-process behaviour or alter resource limits. The implementation selects the boundary at which each operation exposes useful evidence.

Cross-process control is represented by `ptrace` and `security_task_kill`. The `ptrace` entry program retains the request code, target PID and the address and data arguments supplied by the caller. The pointer values remain opaque; the eBPF program does not copy their referenced memory. At exit, the return value is joined with the request so that user space can distinguish acceptance from an error.

Signal delivery is observed through `security_task_kill`. The common event context already identifies the source task, while the event-specific payload describes the destination through its host PID and TID, UID and command name. The signal number completes the source–operation–target relation. Signal zero is retained because it can be used to test whether a target exists and whether the caller has permission to signal it, even though no signal is actually delivered.

Listing 4.4 shows how the destination identity is extracted from the target task rather than inferred from a caller-supplied numeric PID.

```
1  u32 target_pid = get_task_host_tgid(target);
2  u32 target_tid = get_task_host_pid(target);
3  u32 target_uid = 0;
4  char target_comm[TASK_COMM_LEN] = {};
5
6  const struct cred *target_cred =
7      get_task_real_cred(target);
8  if (target_cred)
9      target_uid = BPF_CORE_READ(target_cred, uid.val);
10
11  if (bpf_probe_read_kernel_str(
12      target_comm, sizeof(target_comm),
13      target->comm) <= 0)
14      return 0;
15
16  save_to_submit_buf(&p.event->args_buf,
17                    &target_pid, sizeof(u32), 0);
18  save_to_submit_buf(&p.event->args_buf,
19                    &target_tid, sizeof(u32), 1);
20  save_to_submit_buf(&p.event->args_buf,
21                    &target_uid, sizeof(u32), 2);
22  save_str_to_buf(&p.event->args_buf, target_comm, 3);
23  save_to_submit_buf(&p.event->args_buf,
24                    &sig, sizeof(int), 4);
```

Listing 4.4: Target evidence collected by `security_task_kill` (abridged).

Reading the destination task structure provides more context than a numeric PID supplied by the caller. The hook still runs during mediation: it records the attempted relation, not the final decision or confirmed signal delivery.

Changes to the current task’s behaviour are monitored through `security_task_prctl` and `do_sigaction`. The former records the requested `prctl` option and its four machine-word arguments. The eBPF program intentionally leaves option-specific interpretation to user space, avoiding a large command-dependent

parser in kernel space.

The `do_sigaction` event focuses on signal-handler state. It records the signal number, whether a new action was supplied and whether the caller requested the previous action. For both the proposed and currently installed actions, it captures the flags, first signal-mask word and handler category or address. This makes changes such as installing, replacing or ignoring a handler observable. Since the hook is attached at function entry, the event does not confirm that the proposed action was installed.

Resource-limit monitoring combines mediation with a syscall outcome. `security_task_setrlimit` receives the resolved target task and proposed kernel `rlimit` structure. It records the target host PID, resource identifier, soft limit and hard limit while the operation is being checked. It does not include the final decision.

The paired `prlimit64` event preserves the target PID, resource identifier, addresses of the new and old user-space limit structures, and the syscall return value. The pointed-to structures are not dereferenced by this event. The security hook exposes proposed kernel values; the syscall event preserves the request interface and reported outcome.

`security_settime64` observes attempts to modify the system clock. It records the proposed seconds and nanoseconds together with the timezone offset and daylight-saving field. Wall-clock changes are privileged and can disrupt timestamp analysis or time-based authentication. As a mediation-stage record, however, the event does not show that the proposed time was installed.

4.2.4 File and Filesystem Security

Filesystem events distinguish the name supplied by a process from the object reached by the kernel. A pathname may be relative, use a directory file descriptor or contain symbolic links; the Virtual File System resolves these elements before operating on the object [51].

Syscall probes preserve the original request and returned result. Security hooks see the object after pathname resolution. Its device and inode numbers provide a local identity, since an inode is unique only within its filesystem [24].

The `open` event provides the syscall view. Entry probes for `open` and `openat` retain the operation variant, directory file descriptor, requested pathname,

flags and creation mode. Exit probes add either the returned file descriptor or a negative error. The two syscall variants share one logical schema while retaining their different calling conventions.

The same design is used for filesystem metadata changes. The `chmod` event monitors requests to modify permission bits, which determine whether an object may be read, written or executed. It combines `chmod`, `fchmod` and `fchmodat`, recording the relevant pathname, descriptor or directory-relative context, the requested mode and the syscall result.

The `chown` event monitors requests to replace the user or group ownership of an object. It combines `chown`, `fchown`, `fchownat` and `lchown`, retaining the requested owner and group identifiers, pathname or descriptor context, flags and returned result. The operation identifier remains important because `lchown` changes the ownership of a symbolic link itself rather than following the link to its target.

Six entry and exit programs produce the single `chmod` event, and eight produce `chown`. An operation field identifies the originating syscall, avoiding duplicate public schemas.

`security_file_open` complements the syscall view. It runs after the Virtual File System has resolved the pathname and built a `struct file`. Linux Security Modules may use this point to preserve open-time state and re-evaluate conditions previously checked through `inode_permission` [54]. The event exposes the resolved object under inspection, not the final LSM decision or syscall result.

From the supplied `struct file`, the program obtains the resolved pathname, file flags, device number, inode number and change time. When the originating syscall is recognised, it also attempts to recover the pathname argument from the current task register state. Listing 4.5 shows the essential separation between these two forms of evidence.

```
1 void *resolved = get_path_str(&file->f_path);
2 dev_t device = get_dev_from_file(file);
3 unsigned long inode = get_inode_nr_from_file(file);
4
5 void *requested = "";
6 struct pt_regs *regs = get_current_task_pt_regs();
7
8 if (p.event->context.syscall == SYSCALL_OPEN)
9     requested = get_syscall_arg1(p.event->task, regs, false);
10 else if (p.event->context.syscall == SYSCALL_OPENAT)
11     requested = get_syscall_arg2(p.event->task, regs, false);
```

```
12
13 save_str_to_buf(&p.event->args_buf, resolved, 0);
14 save_to_submit_buf(&p.event->args_buf, &device, sizeof(device), 2);
15 save_to_submit_buf(&p.event->args_buf, &inode, sizeof(inode), 3);
16 save_str_to_buf(&p.event->args_buf, requested, 5);
```

Listing 4.5: Requested and resolved evidence collected by `security_file_open`.

The requested path shows how the process referred to the file; the resolved path is reconstructed from the kernel object. Symbolic links, relative traversal and mount topology may make them differ. Requested-path recovery is best-effort and may be empty when the syscall or register state cannot be identified. The resolved object remains available from `struct file`.

Path reconstruction walks kernel dentries backwards and assembles the result in per-CPU scratch storage. The traversal is limited to 20 components and a 4096-byte working region so that the verifier can prove finite execution and memory access. Deep or disconnected paths may be partial or explicitly marked as disconnected; the result is bounded kernel evidence, not an unconditional canonical path.

A `device:inode` pair is a more stable local correlation key than a path string. It can associate observations that use different paths for the same object. Correlation remains local and time-bounded because inodes can be reused and layered filesystems may expose different identities for apparently equivalent paths.

Other file hooks observe checks and control requests on resolved objects. `security_file_permission` records the pathname, device-inode identity, change time and permission mask presented during a read, write or execution check. It shows that the check occurred, but does not establish whether access was ultimately granted. This is also a high-volume observation point, making selective event activation particularly important.

The `security_file_ioctl` event monitors file and device-specific control requests. It records the resolved target identity, ioctl command and argument value. The argument is intentionally retained as an opaque value: its interpretation depends on both the command and the target device, and decoding every possible structure inside eBPF would increase complexity and verifier cost.

Directory-entry hooks preserve objects at the point where their namespace representation may change. `security_inode_unlink` records the path, de-

vice, inode and change time of an entry proposed for removal. Capturing this identity before the unlink operation is valuable because the pathname may no longer resolve afterwards.

security_inode_rename records the old and new paths, rename flags and source-object identity. Both paths use the same per-CPU scratch buffer, so the source must be serialised before destination reconstruction overwrites it.

Creation-related hooks retain evidence specific to the new directory entry. **security_inode_symlink** records the pathname of the symbolic link and the caller-supplied target string, which may be relative or may not yet identify an existing object. **security_inode_mknod** records the pathname, requested mode and device number of a new special node. The mode identifies the object type and permission bits, while the device number is meaningful for character and block devices. Both events observe security mediation and do not contain a final syscall result.

Mount hooks extend monitoring from individual objects to filesystem topology. **security_sb_mount** records the source name, resolved target path, filesystem type and mount flags. The filesystem-specific data argument is not copied because its format varies between filesystems and may contain sensitive configuration. **security_sb_umount** records the resolved mount-point, filesystem type and unmount flags. As entry observations of security hooks, these events show that the requested topology change reached mediation, not that it completed successfully.

Filesystem records retain three different facts where available: the process request, the resolved object under mediation and the returned result. Neither a pathname request nor an LSM entry observation is treated as proof of success.

4.2.5 Memory, Namespace and Cgroup Transitions

Memory mappings govern address-space access and permissions. Namespaces shape the kernel resources visible to a process, and cgroups place it in a resource control and accounting hierarchy. Their events cover code loading, cross-process memory access and changes to local isolation boundaries.

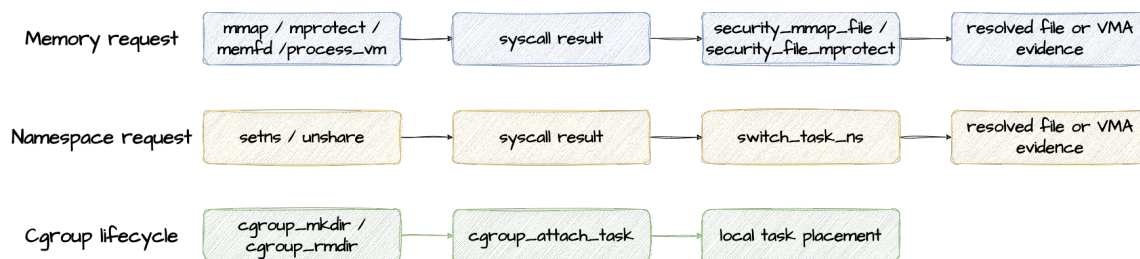


Figure 4.3: Observation points used for memory operations, namespace transitions and local cgroup changes.

Figure 4.3 shows complementary paths, not a mandatory sequence. A memory syscall may not reach its security hook, and a successful namespace syscall need not change every recorded namespace.

The `memfd_create` event records creation of an anonymous, RAM-backed file. Such an object initially has no conventional filesystem pathname, but it may later be mapped into memory or used as an executable image. The event records the requested name and creation flags, followed by the returned file descriptor or a negative error. This makes successful anonymous-file creation distinguishable from a failed attempt without requiring the monitor to inspect its future contents.

The `mmap` event records requests to create or modify mappings in a process address space. Its entry probe retains the requested address, length, protection mask, mapping flags, file descriptor and offset. The exit probe adds either the address selected by the kernel or a negative error. These fields distinguish anonymous mappings from file-backed mappings and identify requests that combine writable and executable permissions.

The `mprotect` event monitors requests to change the permissions of an existing memory range. It records the starting address, length, requested protection mask and returned status. The related `pkey_mprotect` event additionally records the protection key assigned to the range. Protection-key support is dependent on the target architecture and kernel configuration, so the returned value remains necessary for distinguishing an applied request from an unsupported or rejected operation.

Syscall events describe the request but not the kernel objects examined during security mediation. The `security_mmap_file` hook complements `mmap` for file-backed mappings. It records the resolved pathname, file flags, device,

inode, change time, requested protections and mapping flags. Anonymous mappings are omitted because no backing `struct file` is available; they remain visible through the syscall event.

The `security_file_mprotect` hook observes a protection change through the virtual memory area supplied by the kernel. It preserves three distinct protection values: the permissions requested by user space, the permissions adjusted by the kernel for security evaluation, and the access bits currently installed on the region. It also records the region start and length. When the mapping is file-backed, the event adds its resolved pathname and device-inode identity; anonymous regions retain the protection and address evidence without file metadata.

This can show that a writable region is being considered for executable access and whether it belongs to a resolved file. Because the probe runs at security hook entry, it contains neither the hook decision nor the final syscall result.

The program reads the supplied virtual memory area directly instead of searching the process memory-management tree again. This keeps collection bounded and avoids repeating work already done by the kernel.

Cross-process memory access is monitored through `process_vm_readv` and `process_vm_writev`. These events record the target PID, addresses and element counts of the local and remote I/O-vector arrays, operation flags and the number of bytes transferred or the returned error. The vector elements and transferred memory contents are not copied. The events retain the transfer relation without collecting potentially sensitive payload data.

Namespace events monitor changes to the resource views associated with a process. The `setns` syscall requests that the caller join an existing namespace referenced by a file descriptor. Its event records the descriptor, the requested namespace type and the returned result. A type value of zero allows the kernel to infer the namespace type from the descriptor.

The `unshare` syscall requests that selected execution-context resources cease to be shared with other processes. Its event records the complete `CLONE_*` flag mask and the syscall result. In combination, the two events distinguish joining an existing namespace from separating an execution context.

A successful syscall does not expose the identifiers installed in the task. `switch_task_ns` provides this state view at `switch_task_namespaces`, where the kernel replaces the namespace proxy.

The hook compares the old and proposed namespace sets and emits an event only when at least one monitored identifier changes. The record contains the task PID and only the new identifiers for the changed mount, PID-for-children, UTS, IPC, network or cgroup namespaces. Unchanged fields are omitted from the indexed argument stream, keeping the record sparse and suppressing no-op transitions. User namespaces are not part of this namespace-proxy comparison; credential-related user-namespace changes are observed separately through the credential hooks.

Cgroup monitoring covers task placement, not namespace replacement. A cgroup namespace changes how the hierarchy is presented; attachment changes the node to which the task belongs.

The `cgroup_attach_task` raw tracepoint reports movement of a task to a destination cgroup. It records the destination cgroup and hierarchy identifiers, the supplied cgroup path, target command name, host PID and TID, and whether the operation concerns the complete thread group. This allows the monitor to observe containment changes even when the task identity itself remains unchanged.

The `cgroup_mkdir` and `cgroup_rmdir` events describe the lifecycle of the local cgroup hierarchy. Both retain the cgroup identifier, bounded path and hierarchy identifier supplied at creation or removal. Each is a local lifecycle observation, not a persistent hierarchy.

Cgroup identifier extraction also contains a compatibility decision specific to the supported kernel range. The layout of the underlying `kernfs_node` identifier differs between older kernels, newer kernels and the hidden KABI representation used by the target RHEL-derived kernel. CO-RE field-existence checks select the available layout at load time, allowing the same helper to obtain the identifier without hard-coding a single structure version.

Namespace inode numbers and cgroup identifiers are local kernel identities. They support comparison and detector correlation on one node, but are not pod, container or cluster-wide workload identifiers. No persistent namespace graph, cgroup hierarchy or Kubernetes execution model is reconstructed.

4.2.6 Kernel Integrity and Kernel-Facing Activity

Kernel modules, eBPF objects and dynamic instrumentation can inspect or change kernel behaviour. Their interfaces are legitimate but security-sensitive. The implemented hooks record the operation, object identity and

result available at each observation point; classification is left to detectors.

Kernel-module monitoring separates lifecycle notifications from initialisation results. The `module_load` raw tracepoint records a module as it enters the kernel module lifecycle, while `module_free` reports its removal. Both events preserve the module name and, when available, its version and source-version strings. They do not copy module code, sections or other internal contents.

Lifecycle events do not show whether initialisation succeeded. `do_init_module` therefore uses a kprobe and kretprobe pair. At entry, the module object is retained in per-thread temporary state. At return, the implementation retrieves that object, extracts the module identity and appends the function result. A zero value identifies successful initialisation, while a negative value preserves the reported kernel error.

The tracepoints identify additions and removals; `do_init_module` provides the initialisation outcome. Seeing a module during loading is therefore not treated as proof that it became operational.

The eBPF hooks are kprobes on Linux Security Module functions used during security checks for eBPF commands and objects. They are not BPF LSM enforcement programs and do not change the decision.

The `security_bpf` hook monitors operations entering the security path of the BPF syscall. Its purpose is to expose requests such as map creation, program loading and program attachment. The event records the BPF command and attribute size, but does not copy `union BPF_attr`, whose active fields depend on the command.

The `security_bpf_map` hook observes security checks involving an existing eBPF map. Since maps retain state and exchange data between kernel programs and user space, identifying access to them provides useful object-level evidence. The event records the map identifier and bounded name, without collecting its contents.

The `security_bpf_prog` hook observes security operations involving an existing eBPF program. It records the program identifier, name and type, allowing programs intended for kprobes, tracepoints, cgroups and other execution contexts to be distinguished.

These hooks expose the requested BPF operation and the map or program under security mediation.

Further events cover kernel-facing activity that ordinary process syscalls may not reveal: dynamic tracing, symbol lookup, procfs creation, kernel-initiated

execution and kernel file consumption.

The **register_kprobe** event monitors attempts to install dynamic kprobes. It records the target symbol, pre- and post-handler addresses, and registration result. This distinguishes successful installations from rejected requests, although the symbol may be absent when registration uses a kernel address directly.

The **kallsyms_lookup_name** event records kernel symbol resolution. It combines the requested symbol name with the returned address, where zero indicates that no address was found. It covers this specific lookup interface rather than every possible source of kernel addresses.

The **proc_create** event records requests to create procfs interfaces. It records the entry name and proc-operations address, making kernel-created user-space control or information endpoints visible. It does not include the parent directory, complete path or creation result.

The **call_usermodehelper** event records requests by the kernel to start a user-space program. It records the executable path, bounded argument and environment arrays, and wait mode. The arrays contain at most 38 elements and remain subject to the event-size limit. The event represents the request, not successful execution or completion of the helper.

The **security_kernel_read_file** event observes the security stage before the kernel consumes a file. It records the resolved pathname, device, inode, change time and read purpose, distinguishing uses such as modules, firmware, policy data and kexec images.

The later **security_kernel_post_read_file** event records the same file identity and purpose together with the supplied byte size. This size is not a return value. The two stages are emitted independently: the implementation does not correlate them automatically, copy file contents or compute a digest.

The resulting coverage does not inspect handler code, reconstruct every kernel relationship, prove integrity or enforce a decision. Chapter 5 explains how these logical events activate their producers and support dependencies.

4.3 Binary Event Contract and Kernel Transport

Every event uses the same binary format. A 136-byte context holds common process, host and event metadata, followed by a one-byte argument count and the captured arguments. The Go decoder mirrors the C field order and

widths. Only the populated prefix is submitted, not the entire 32,000-byte construction workspace.

Each argument starts with its one-byte schema index. Fixed-width values follow directly; variable-length values add a four-byte length. Strings and argument arrays contain bounded sub-records. No wire type tag is needed because the index selects the name and decoder from the Go registry. Missing arguments can be omitted without renumbering later fields.

Before writing an argument, the eBPF helper must prove to the verifier that the current offset and the complete payload remain inside the buffer. Listing 4.6 shows the essential checks.

```
1  if (size == 0 || size > MAX_ELEMENT_SIZE || buf->offset >=
    ↪ ARGS_BUF_SIZE)
2      return 0;
3  u32 offset = buf->offset;
4  update_min(offset, ARGS_BUF_SIZE - 1);
5  u32 buffer_index_offset = offset + 1;
6  if (buffer_index_offset > ARGS_BUF_SIZE)
7      return 0;
8  buf->args[offset] = index;
9  // The omitted block clamps size for the older verifier.
10 u32 buffer_arg_offset = buffer_index_offset + size;
11 if (buffer_arg_offset > ARGS_BUF_SIZE)
12     return 0;
13 if (bpf_probe_read(&(buf->args[buffer_index_offset]), size, ptr) < 0)
14     return 0;
15 buf->offset = buffer_arg_offset;
16 buf->argnum++;
17 return 1;
```

Listing 4.6: Checked argument-buffer offset handling.

The helper checks the original offset and uses `update_min` only to make its bound visible to the verifier. The stored offset advances after the index and complete payload have been copied.

Once the event is complete, the submission helper refreshes its task context, optionally adds a user-space stack identifier and calculates the record size. It sends the record through `BPF_perf_event_output` to the perf-event buffer associated with the current CPU. The Go runtime consumes these per-CPU buffers through a libbpfgo perf reader; the current implementation does not retain a BPF ring-buffer transport path.

The reader's loss channel reports records dropped by a full per-CPU buffer;

malformed records that arrive are reported by the decoder. These remain separate conditions. Section 3.4.2 covers the transport’s capacity and ordering consequences.

4.4 Kernel-Side UID Filtering

The shared initialiser can filter on one exact effective UID. When enabled, it reads the current task’s effective UID and stops a mismatch before full context enrichment and perf-buffer submission. Because the check is in the common path, it applies to every producer using the framework from Section 4.2.1.

The mechanism supports one equality comparison, not ranges, exclusion sets, namespace-aware identities or policy expressions. A missing configuration or disabled flag admits the event. A rejected observation creates no event or authorisation result. Richer selection stays in user space.

Before processing begins, the Go runtime writes the enable flag and UID to fixed fields in the shared configuration map. These fields must remain aligned with the C structure. Chapter 5 covers the update lifecycle; this section is limited to kernel-side admission.

4.5 Compatibility and Verifier Challenges

CO-RE adapts field access when compatible BTF is available, but some kernel differences still need explicit branches. The shared helpers cover alternative task identifiers and start times, capability layouts and inode change times. Cgroup identification includes the internal layout used by the RHEL 8 kernel family. CO-RE selects branches whose fields exist on the target.

Other paths depend on helper availability. Syscall registers use BTF-aware task helpers when available; otherwise, an x86-64 fallback derives the frame from the task stack using the target’s expected page and stack dimensions. Compatible 32-bit processes have their syscall numbers translated before event classification. A bounded red-black-tree VMA lookup is retained for older kernels, but the memory hooks in Section 4.2.5 do not call it.

Verifier constraints also shape the code. Path reconstruction, string arrays and VMA traversal use fixed loop bounds. Buffer writes and submitted sizes

repeat checks in forms accepted by the target verifier. The shared syscall-exit path limits the return-argument index to values below 16 because the broader bound was rejected on Rocky Linux. Selected memory syscalls use typed tracepoints instead of the more complex generic dispatcher for the same reason.

These branches let one object handle compatible layouts on the verified target; they cannot provide missing symbols, hooks, helpers or attachment mechanisms. The portability claim remains limited to the identified Rocky Linux environment, in line with Section 2.2.5.

4.6 Kernel-Space Implementation Summary

Direct, paired and lifecycle producers share one bounded construction framework. Their records keep requests, security mediation, committed transitions and returned outcomes distinct. Indexed arguments allow unavailable fields to be omitted without changing schema identity, and the completed record crosses the perf buffer after optional exact UID admission.

The object is tailored to the verified Rocky Linux target. CO-RE handles compatible layout differences; explicit branches and verifier-visible bounds cover older and vendor-specific details. Chapter 5 continues from the point at which user space selects, loads and consumes these producers.

Chapter 5

User-Space Runtime and Detection Implementation

Chapter 4 followed an event to the perf-buffer boundary. This chapter continues in user space, where the Go runtime prepares the selected programs, decodes their records, applies policies and detectors, and delivers events, alerts and diagnostics.

5.1 Command-Line Configuration and Runtime Bootstrap

The user-space workflow begins with a Cobra command-line interface [28]. It parses operator input into one configuration object and invokes the application entry point. Policy loading, detector evaluation and eBPF attachment remain outside Cobra.

Defaults provide the host BTF path, JSON event and alert output, console diagnostics and an informational log level. CLI flags update event selection, kernel filtering, policy and detector paths, output, logging and eBPF input. The resulting typed structure is passed through the runtime instead of a loose set of command-line values.

Configuration is normalised and validated before kernel resources are acquired. Detector paths enable detector processing, and **alerts-only** enables alert generation while suppressing ordinary event output. Unsupported event, alert or logging formats fail at this stage, before loading or

attachment.

The `list-events` path reads public names from the probe registry and exits without resolving BTF, opening the eBPF object or attaching programs. A monitoring invocation resolves the object, creates the runner and hands over the validated configuration.

Listing 5.1 shows this handoff.

```
1  if err := ValidateConfig(&cliCfg); err != nil {
2      return err
3  }
4  if cliCfg.ListEvents {
5      printSupportedEvents(cmd.OutOrStdout())
6      return nil
7  }
8  if err := initialize.BPFObject(&cliCfg); err != nil {
9      return err
10 }
11 runner, err := NewProjectRunner(cliCfg)
12 if err != nil {
13     return err
14 }
15 return runner.Run(ctx)
16
```

Listing 5.1: Validated handoff from the Cobra command to the runtime (abridged).

Static configuration, external-definition and construction errors are reported before monitoring starts. Failures confined to one record are handled later by the active event loop.

5.2 Loader, Probe Registry and Attachment Lifecycle

After validation, the bootstrap phase resolves the artefacts required to load the eBPF programs. The eBPF object is an ELF file produced by compiling the C collector sources.

The runtime first checks the `PROJECT_BPF_OBJECT` environment variable, followed by the path supplied through the command-line interface. If neither is specified, it reads `dist/project.bpf.o`, which is embedded in the Go executable at build time. In every case, the selected object is loaded into

memory and passed to `libbpfgo` as a byte buffer.

The BTF file is resolved separately. The `PROJECT_BTF_FILE` environment variable takes precedence over the command-line configuration, whose default is `/sys/kernel/btf/vmlinux`. This file describes the types of the running kernel and allows `libbpf` to apply the CO-RE relocations required by the compiled eBPF object.

Policies and detectors are prepared before the eBPF module is opened. A disjoint finite policy and CLI selection fails startup, avoiding an empty set being interpreted as “all events”. Selecting a detector does not enable its kernel events independently: its inputs must already be in scope.

The probe registry converts logical events into producer programs and any support probes needed to retain intermediate information.

Listing 5.2 shows this dependency expansion.

```
1   for _, probe := range supportedProbes {
2       if probeActivation[probe.eventName] {
3           selectedProbes = append(selectedProbes, probe)
4           continue
5       }
6       for _, event := range probe.impliedBy {
7           if probeActivation[event] {
8               selectedProbes = append(selectedProbes, probe)
9               break
10          }
11      }
12  }
13
```

Listing 5.2: Selection of producer and support probes (abridged).

Registry tests verify the expected producers and support probes. Hook availability can only be established by loading and attaching on the target system.

Before object loading, autoload is disabled for registered programs outside the plan. `Libbpf` then verifies and loads the selected producers and dependencies. The optimisation covers only programs represented in the registry.

After loading, the runtime updates the required maps, creates the perf-event reader and attaches each selected program. Monitoring ends when the context is cancelled, normally by `SIGINT` or `SIGTERM`.

5.3 User-Space Decoder and Event Registry

The perf reader receives each kernel record as a byte slice. The decoder checks its structure and converts it into the typed representation used by the Go runtime.

Figure 5.1 shows the regions read by the decoder: a 136-byte common context, the argument count at byte 136 and the variable stream of indexed arguments.

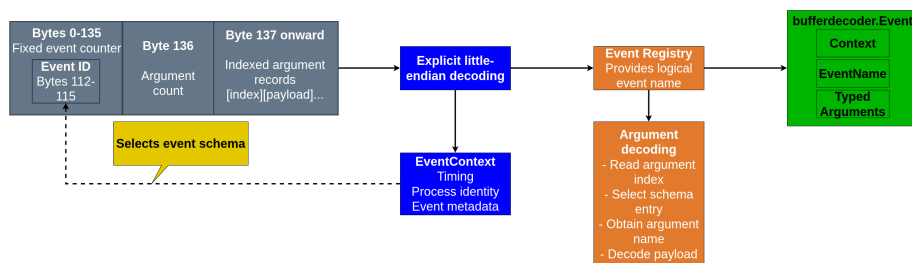


Figure 5.1: Binary record layout consumed by the user-space decoder.

The fixed context must be complete before any field is read. Header values are decoded from explicit little-endian offsets rather than copied into a Go structure, whose compiler-defined padding could differ from the C wire layout.

The resulting context includes timing, process and namespace identities, command and UTS names, the event identifier, associated syscall and collection metadata. The event identifier selects an entry in the user-space registry.

Each registry entry associates a public identifier with a logical name and an ordered argument schema. The schema supplies the name and type omitted from the wire representation.

An argument begins with a one-byte schema index followed by its payload. The selected type determines the decoder: scalars have fixed widths; strings, byte sequences and arrays carry bounded lengths; socket addresses and credentials use dedicated structured decoders.

```

1 spec, ok := events.GetSpec(eventID)
2 if !ok {
3     return nil, fmt.Errorf("unknown event id=%d", eventID)
4 }

```

```
5
6   for seen := 0; seen < argnum && offset < len(rawArgs); seen++ {
7       index := int(rawArgs[offset])
8       offset++
9
10      if index >= len(spec.Args) {
11          return nil, fmt.Errorf("argument index out of range")
12      }
13
14      argSpec := spec.Args[index]
15      value, consumed, err :=
16          decodeArgRecord(rawArgs[offset:], argSpec.DecodeAs)
17      if err != nil {
18          return nil, err
19      }
20
21      offset += consumed
22      decoded[index] = Argument{
23          Name:  argSpec.Name,
24          Type:  argSpec.DecodeAs,
25          Value: value,
26      }
27  }
28
```

Listing 5.3: Schema-directed decoding of indexed event arguments.

A valid record becomes a `bufferdecoder.Event` containing the common context, public event name and typed arguments. Policies, detectors and output all consume this representation; presentation does not perform another normalisation step.

5.4 User-Space Event Selection and Admission

After decoding, three short-circuit gates check the logical event set, an optional exact command name and the active policies. A policy applies its common scope before its rules, and matching suppression overrides monitor or detect rules. Without policies, the last gate admits the event.

Listing 5.4 shows the order and consumer fan-out. Optional socket enrichment and diagnostics are omitted.

```
1  event, err := bufferdecoder.DecodeEvent(raw)
2  if err != nil {
3      return
```

```
4 }
5 if !p.eventEnabled(event.EventName) {
6     return
7 }
8 if !p.commEnabled(event.Context.CommString()) {
9     return
10 }
11 if !p.policyEnabled(event) {
12     return
13 }
14 if !p.cfg.Alerts.Only {
15     if err := p.printer.Print(event); err != nil {
16         p.logger.Error("print event", zap.Error(err))
17     }
18 }
19 p.processDetectors(ctx, event)
```

Listing 5.4: Decoded-event admission and consumer fan-out (abridged).

Failure at any gate prevents both event output and detector evaluation.

Valid rejections are distinguished from failures. Only the first few filter drops are logged at debug level, preventing unbounded diagnostics. Decode, detector and rendering errors are reported separately without stopping the loop. Component tests cover these paths, although no unit test exercises the complete admission order end to end.

5.5 Policy and Detector Engine

5.5.1 YAML Policy and Detector Definitions

Policies and detectors use separate YAML documents, both validated and converted into Go models before event processing.

Listing 5.5 shows the main policy fields.

```
1 name: privileged-file-monitoring
2 mode: detect
3 intent: threat_hunting
4
5 scope:
6     uids: [0]
7
8 rules:
9     - name: sensitive-files
```

```
10     events:
11         include: [security_file_open]
12         filters: ["args.pathname=/etc/*"]
13
14     detectors:
15         include:
16             severities: [high, critical]
17             techniques: [T1548]
18             stateful: true
19         exclude:
20             tags: [experimental]
21
```

Listing 5.5: Representative YAML policy structure.

The **name**, **mode** and **intent** identify the policy and its purpose. Mode and intent default to **monitor** and **threat_hunting**. An optional **scope** applies command or UID constraints to all rules.

Rules are alternatives; fields within one rule are cumulative constraints. Event exclusions override inclusions, and an empty include list starts from all events. Filters written as **args.<name>** run after decoding because they need typed arguments.

The optional **detectors** block selects definitions by identifier, severity, tags, statefulness or MITRE ATT&CK metadata. Values in one criterion are alternatives; populated criteria are combined. Exclusions are then removed from the catalogue. Suppression policies cannot select detectors.

A detector document defines input events, matching logic, alert fields and optional ATT&CK classification. Listing 5.6 shows a shortened collective definition.

```
1  id: privilege-exec-chain
2  name: Privilege change followed by execution
3  requires:
4      - {event: security_task_fix_setuid,
5        reason: observes a UID transition}
6      - {event: sched_process_exec,
7        reason: confirms executable replacement}
8  consumes: [security_task_fix_setuid, sched_process_exec]
9
10 stateful: true
11 window: 5s
12 group_by: [process_tree]
13 steps:
14     - event: security_task_fix_setuid
```

```
15     conditions:
16       - {field: args.new_euid, operator: eq, value: "0"}
17     - event: sched_process_exec
18       conditions:
19         - {field: uid, operator: eq, value: "0"}
20
21     output:
22       title: Privilege change followed by process execution
23       severity: high
24     threat:
25       tactics: [{id: TA0004, name: Privilege Escalation}]
26       techniques: [{id: T1548,
27                     name: Abuse Elevation Control Mechanism}]
28     tags: [process, collective, privilege-escalation]
29
```

Listing 5.6: Representative YAML definition of a collective detector.

The stable `id` supports selection and alert attribution. `requires` documents event dependencies; `consumes` defines the dispatch set and is derived from requirements and steps when omitted.

Stateless predicates appear in top-level `conditions`. Collective definitions use ordered `steps`, a bounded `window` and `group_by` strategies for process, immediate parent, file resource, local cgroup or composite identity. Subsection 5.5.3 describes their state.

The `output` block defines the alert; `threat` carries optional MITRE ATT&CK classification. Conditions, event names, severity, ATT&CK identifiers and correlation plans are validated and compiled at startup.

Definitions may be supplied as files or through directories scanned non-recursively in lexical order.

5.5.2 Point and Collective Evaluation Paths

As introduced in Subsection 3.6.2, point and collective detectors require different amounts of evidence. Both implement the same `Detector` interface and use the event-indexed dispatcher. The YAML-backed implementation chooses the path when it receives an event.

Listing 5.7 shows the branch between immediate matching and sequence processing.

```
1   if !d.consumes(event.EventName) {
```

```
2     return nil, nil
3 }
4
5 if d.isStatefulSequence() {
6     return d.onStatefulEvent(event)
7 }
8
9 matches, err := d.matches(event)
10 if err != nil || !matches {
11     return nil, err
12 }
13
14 return []detectors.Alert{d.newAlert(event)}, nil
15
16 func (d *Detector) isStatefulSequence() bool {
17     return d.def.Stateful && len(d.steps) > 1
18 }
19
```

Listing 5.7: Selection of the point or collective evaluation path.

The consumption check remains at the detector boundary even though dispatch has already narrowed the candidates. The collective path requires both `stateful: true` and more than one compiled step; the flag alone does not create a collective detector.

Collective definitions are delegated to `onStatefulEvent`, which derives correlation keys and tests the next expected step. An event can advance or complete existing state, or start a new sequence by matching its first step.

5.5.3 Compiled Correlation State and Alert Deduplication

YAML `group_by` values are compiled into a correlation plan during detector construction. Event processing then calls preselected key extractors without parsing strategy names again.

The supported components produce these internal values:

- `process` combines the host PID and task start time;
- `process_tree` can additionally produce the immediate parent's host PID and start time when looking up an existing sequence;
- `resource` combines the device and inode arguments;

- `cgroup` uses the local kernel cgroup identifier.

Several components form one composite key, and all must be present. Missing process, resource or cgroup data stops the transition instead of falling back to a weaker identity.

Plan construction rejects unknown or duplicate components. Resource correlation requires `dev` and `inode` in every step. `user_session` is unsupported because events do not expose a stable session identifier; UID is not used as an approximation.

Each collective detector maps correlation keys to partial sequences. An entry stores the next step, initial time and matched evidence. Listing 5.8 shows how an event advances it.

```
1  for _, key := range lookupKeys {
2      state, ok := d.state[key]
3      if !ok {
4          continue
5      }
6      if now.Sub(state.firstSeen) > window {
7          delete(d.state, key)
8          continue
9      }
10
11     matches, err :=
12         stepMatches(event, d.steps[state.nextStep])
13     if err != nil {
14         return nil, err
15     }
16     if !matches {
17         continue
18     }
19
20     state.events = append(state.events, event)
21     state.nextStep++
22
23     if state.nextStep >= len(d.steps) {
24         delete(d.state, key)
25         return []detectors.Alert{
26             d.newAlertFromEvents(state.events),
27         }, nil
28     }
29
30     d.state[key] = state
31     return nil, nil
32 }
```

Listing 5.8: Advancement of a partial collective sequence (abridged).

Process-tree lookup may use both the current process and its immediate parent, allowing a child event to advance parent state without a persistent process graph. If no state advances, a first-step match creates an entry under its initial key. Each key retains at most one partial sequence.

Table 5.1 lists the state safeguards; they are not performance results.

Mechanism	Value	Runtime effect
Stateful window	2-second default; 5-second maximum	Bounds the lifetime of incomplete evidence. Longer definitions are rejected during validation.
Periodic pruning	<code>min(window/2, 1 second)</code>	Avoids scanning the complete state map for every event. The directly addressed entry is still checked immediately.
Sequence capacity	4,096 keys per detector	Inserting a new key at capacity evicts the oldest incomplete sequence.
Alert deduplication	5-second suppression window	Prevents repeated equivalent alerts from being emitted during the configured interval. Entries older than 10 seconds are removed when later alert batches are processed.

Table 5.1: Runtime bounds for collective state and alert deduplication.

Expired state is removed when addressed or during periodic pruning triggered by relevant events. The interface exposes `Flush`, but the production runtime does not schedule it independently. Idle stale entries may remain until later traffic or capacity pressure triggers cleanup.

Alert deduplication runs after evaluation and is independent of sequence state. Its key contains detector identity, title, severity and fields from the first evidence event, excluding timestamps and thread identifiers. Repeated keys inside the window are suppressed.

The key is neither the complete collective sequence nor a durable causal identity. The deduplication map also has no independent capacity limit.

5.6 Output and Structured Logging

The output package formats the decoded model; it does not normalise it again. Events become JSON or compact text, while alerts add detector identity, severity, ordered evidence and optional threat metadata. JSON preserves typed values and can add readable labels; compact text supports interactive use.

Events and alerts both use standard output but have independent format settings, so one run can produce a mixed stream. `alerts-only` suppresses event presentation without disabling detection. Point alerts contain one source event; collective alerts preserve ordered evidence in JSON and a bounded sequence in compact text.

Zap writes operational diagnostics to standard error in console or JSON form at debug, info, warn or error level [63]. These include lifecycle and selected libbpf messages, perf-buffer losses, and decoder, detector or output failures. Zap never serialises monitored events or alerts, keeping infrastructure conditions outside the security data stream.

Class	Source model	Channel	Failure behaviour
Normalised event	bufferdecoder.Event through a printer	stdout	A write failure is diagnosed; subsequent events continue.
Detector alert	Detector alert with one event or ordered evidence	stdout	A rendering failure is diagnosed; it does not change detector state.
Operational diagnostic	Runtime, decoder, detector, output or libbpf condition	stderr through Zap	Record-local conditions are logged without becoming security events.
Fatal bootstrap or lifecycle error	Command, definition loading or runtime setup return value	stderr through the command path	Startup or shutdown reports an error; cleanup errors are joined where available.

Table 5.2: User-space output and diagnostic boundaries.

Table 5.2 summarises channels and failure behaviour. Configuration and startup errors return to the command entry point; most record-local failures are logged and processing continues.

Chapter 6

Experimental Evaluation

Chapter 5 completed the implementation account at the point where decoded events are admitted, evaluated and rendered. This chapter evaluates that pipeline in a controlled containerised scenario. A vulnerable service, an exploit client and the monitor run in three separate Pods on one Minikube node. The experiment asks whether the node-level monitor can observe activity in the target Pod, whether the configured collective detector preserves the expected evidence, and whether exact kernel-side UID admission keeps the monitor’s measured CPU demand below the project’s 5% threshold.

The evaluation is deliberately bounded. It is not a measurement of general detection accuracy, cross-node visibility or total system overhead. Instead, it combines a functional campaign of three benign and three exploit runs with a performance campaign of six monitored runs comparing two configurations. Application outcome, detector outcome, operational health and resource consumption are treated as separate observations throughout.

6.1 Evaluation Objectives and Experimental Protocol

The experimental protocol addresses three objectives.

- First, it tests node-local, cross-Pod visibility: the monitor must receive target events even though the monitor, target and trigger have different Pod identities, network addresses and cgroups.

- Second, it tests the scenario-specific collective detector. The exploit should produce one alert containing four ordered events, whereas the selected benign interaction should not complete that sequence.
- Third, it compares the monitor’s resource use with exact UID admission enabled and disabled while retaining the same exploit workload and detector outcome.

These objectives require independent evidence oracles. Application success is established by the benign response or by recovery of the synthetic flag. Detector success requires one alert with the complete ordered evidence. Operational health is assessed from Pod restart deltas and perf-event loss diagnostics emitted during each retained interval. The exploit client also performs its own output validation, but this is a harness check rather than a substitute for either the application or detector oracle. This distinction is important because two retained exploit runs recovered the flag despite a defective client-side pattern check.

For the functional campaign, the independent variable is the interaction class, benign or exploit. The dependent observations are the application result, client result, alert count and completeness, reported loss diagnostics and restart delta. Three runs of each class were retained without replacement. For the performance campaign, the independent variable is the monitor profile: **exact UID-filtered or unfiltered**. The primary dependent metric is monitor-container cgroup CPU consumption; cgroup memory, cgroup `total_rss`, target CPU and node CPU provide additional context. Each profile was repeated three times, with their relative order varied across the three rounds to reduce simple ordering bias.

Each run was assigned an identifier. Before the interaction began, workload readiness and Pod state were verified; after completion, the corresponding trigger output, monitor and target logs, restart counts and, where applicable, resource samples were retained under the same identifier. Effective manifests and the complete command history were preserved with the experimental artefacts. This organisation supports repeatability within the laboratory, although three repetitions per condition are not sufficient for population-level accuracy or overhead claims.

6.2 Kubernetes and Minikube as the Evaluation Platform

The evaluation uses Kubernetes to deploy and coordinate the containerised components involved in the experiment. Kubernetes manages applications through declarative resources that describe their desired execution state. Its basic execution unit is the Pod, which groups one or more containers that share selected namespaces and resources. Containers placed in separate Pods instead retain distinct workload identities and, unless explicitly configured otherwise, separate process and network contexts [36, 39].

This model is relevant to the evaluation because the monitoring prototype and the application under observation can run as independent workloads while still executing on the same node. The application does not need to include the monitoring executable or expose an application-specific telemetry interface. The monitor obtains visibility from the node kernel, whereas Kubernetes controls the placement, isolation and lifecycle of the corresponding containers.

The monitor is deployed through a DaemonSet, a controller intended for node-local services such as monitoring and logging agents. In a multi-node environment, this model can maintain one monitor instance on each eligible node; in the present evaluation, it produces one instance because the cluster contains a single node [34].

Communication with the target is provided by a Kubernetes Service. Pods are ephemeral and may receive different addresses when recreated, whereas a Service provides a stable logical endpoint and selects the Pods that implement the application. The experimental client can therefore address the service without depending on the current IP address of the target Pod [37].

The Kubernetes environment is instantiated locally through Minikube. Minikube provisions a Kubernetes cluster on one machine and supports several drivers for creating its node environment. The experiment uses the Docker driver, under which the Minikube node is hosted through Docker rather than through a dedicated virtual machine [57, 56]. The resulting arrangement preserves the Kubernetes resource and networking model while keeping the testbed reproducible on the selected Rocky Linux host.

Minikube is used here as a controlled evaluation platform. It makes it possible to deploy the monitor, target and experiment client through Kubernetes manifests, inspect their runtime state with `kubectl`, and repeat the scenario

without requiring external cluster infrastructure. The concrete software versions, node resources, workload identities and security settings used in the experiment are reported in Section 6.3.

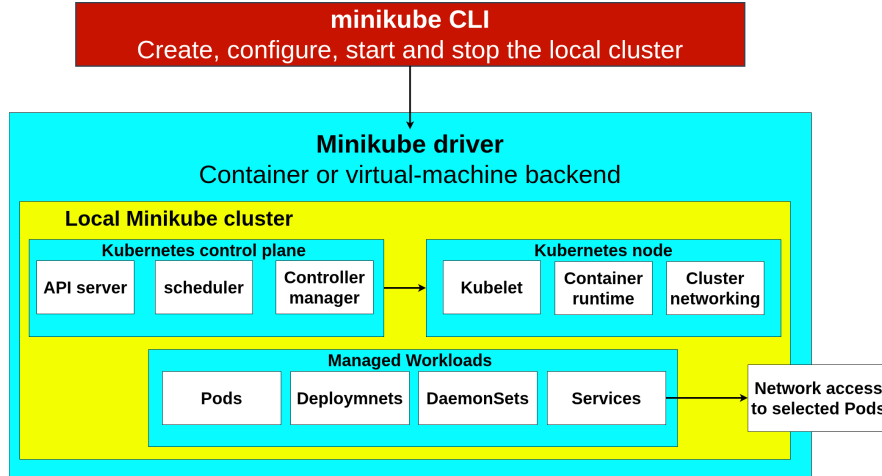


Figure 6.1: General representation of Minikube environment on a single host.

6.3 Cross-Pod Minikube Experimental Testbed

The testbed ran on an x86-64 Rocky Linux 8.10 host. Minikube v1.38.1 used its Docker driver to create one node configured with two CPUs and 4096 MiB of memory [56]. The Kubernetes server was v1.30.0; the host client was v1.36.3. The node reported `docker://29.2.1` as its container runtime, while the host Docker CLI was v26.1.3. The CPU and memory values are the requested Minikube configuration because realised allocatable resources were not preserved as experimental evidence.

Table 6.1 combines the environment and workload roles. All three Pods were scheduled on the same node and therefore shared its kernel, but they did not share a Pod or process namespace. Kubernetes gives Pods separate network identities; the ClusterIP Service instead selected the target endpoint on TCP port 1337 [38]. The observed Service address was 10.102.19.49 and its target endpoint was 10.244.0.3:1337.

Component	Form and identity	Experimental boundary
Host and node	Rocky Linux 8.10; one Docker-backed Minikube node;	Two CPUs and 4096 MiB requested; no multi-node observation.
Monitor	DaemonSet; UID/GID 0; privileged;	host BTF mounted read-only; bpffs and tracefs mounted read-write.
Target	Single-replica Deployment; UID/GID 10001; non-root	All capabilities dropped, privilege escalation disabled, read-only root filesystem; the synthetic flag Secret is mounted only here.
Trigger	Standalone Pod; UID/GID 10002; non-root	Same capability and filesystem restrictions as the target.

Table 6.1: Recorded environment and component boundaries of the cross-Pod testbed.

The monitor’s privileged security context and host-interface mounts provide the access needed by this prototype to load and attach its eBPF programs. They are sufficient laboratory permissions, not a demonstrated least-privilege deployment [35]. Target and trigger instead ran as separate non-root identities, dropped all capabilities and disabled privilege escalation. Service-account token automounting was disabled for all Pods. Resource requests were present, but resource limits were not, leaving the components able to contend for the single node.

Figure 6.2 shows the boundary evaluated by the experiment. The trigger reaches the target through Kubernetes networking; the monitor does not join that application path. It observes the common node kernel from its own Pod. Disabling `hostPID` also avoids giving the application Pods host process visibility merely for the experiment [39].

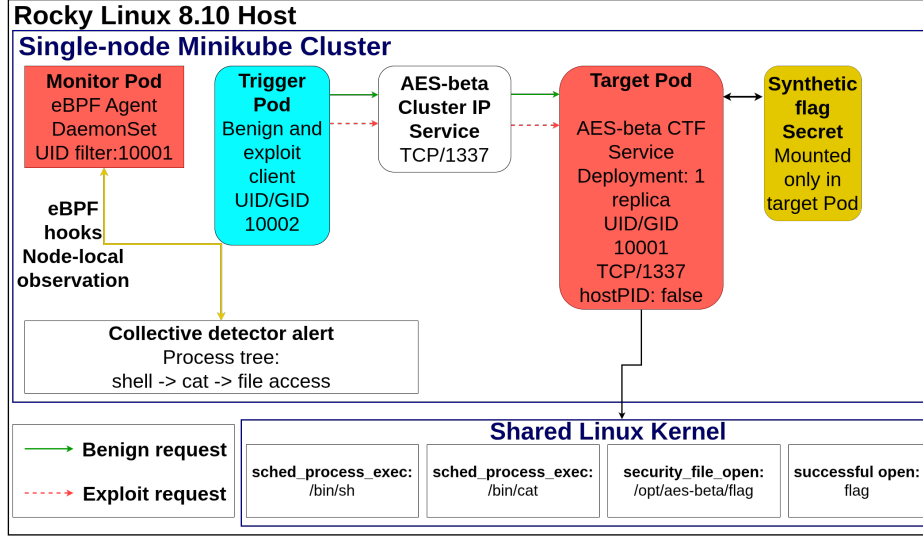


Figure 6.2: Cross-Pod topology on the single Minikube node.

6.4 AES-beta Workload and Controlled Attack Scenario

AES-beta is a public CTF challenge from openECSC-2024 [14]. The experiment used a fixed local copy, adapted only where required for repeatable execution in the laboratory. The original competition secret was replaced by the synthetic value mounted into the target Pod.

The benign client submits a valid key and plaintext, receives the transformed message and compares it with the expected result. It exercises the network service without requesting the flag. The exploit first leaks the information needed to locate the library and reconstructs the stack canary by testing the unknown bytes. Its final worker supplies a payload that invokes `system("cat flag")`. The resulting target-side behaviour installs `/bin/sh`, then `/bin/cat`, and accesses the flag at `/opt/aes-beta/flag`. Each retained exploit reported 1,793 workers and recovered the synthetic flag.

The monitor is not a network intrusion detector and does not inspect the payload, leak or canary search. Its experimental policy admits the selected target events and, in the filtered profile, applies exact effective UID 10001

admission in kernel space. The collective detector requires four observations within five seconds: execution of `/bin/sh`; execution of `/bin/cat`; an LSM `security_file_open` observation resolving `/opt/aes-beta/flag`; and an `open` event for caller pathname `flag` with a non-negative returned file descriptor. The `process_tree` strategy relates those observations locally.

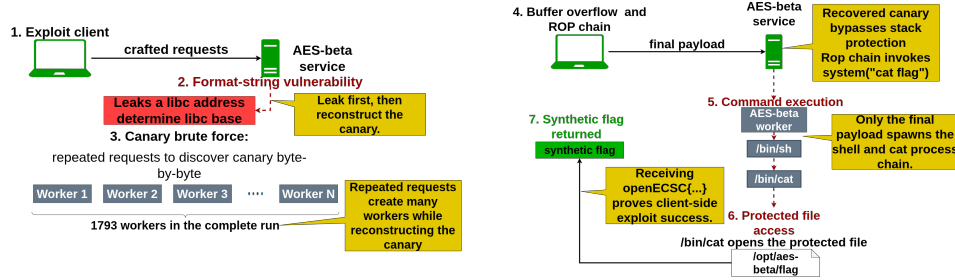


Figure 6.3: Controlled AES-beta exploitation flow. The preparation phase is shown on the left, while the final payload and resulting process chain are shown on the right. Shell execution, `cat` execution and flag access satisfy the configured four-event detector sequence.

The upper panel gives the minimum exploit context; the lower panel owns the transition from final payload to event sequence and alert. The diagrams are explanatory rather than packet or event timelines.

The alert carries ATT&CK tactics TA0002 and TA0009 and techniques T1059.004 and T1005. These identifiers classify the intended meaning of the configured evidence [26].

6.5 Functional Evaluation

6.5.1 Collective Detector Validation

All three exploit runs recovered the synthetic flag and generated exactly one alert. Each alert retained four records in the required order: a `sched_process_exec` event for `/bin/sh`, another for `/bin/cat`, a `security_file_open` record resolving the mounted flag, and an `open` record. The final `open` event completed the evidence by recording the syscall return value. In the retained alert, the value was 2: a non-negative result indicating success and identifying the file descriptor assigned by the kernel.

Within each alert, all four records carried UID 10001 and the same stable host process identity. The experiment therefore exercised the same-process branch of `process_tree` correlation.

The following listing is a field projection of the original JSON alert emitted during exploit run E1. The values are taken directly from the retained monitor log; timestamps and context fields not required to interpret the detector sequence are excluded from the projection.

```
1  {
2    "type": "alert",
3    "detector_id": "aes-beta-command-execution",
4    "severity": "high",
5    "event_count": 4,
6    "events": [
7      {
8        "event_name": "sched_process_exec",
9        "process": {"uid": 10001, "comm": "sh"},
10       "host": {"pid": 3848448},
11       "args": [
12         {"name": "fileName", "value": "/bin/sh"}
13       ]
14     },
15     {
16       "event_name": "sched_process_exec",
17       "process": {"uid": 10001, "comm": "cat"},
18       "host": {"pid": 3848448},
19       "args": [
20         {"name": "fileName", "value": "/bin/cat"}
21       ]
22     },
23     {
24       "event_name": "security_file_open",
25       "process": {"uid": 10001, "comm": "cat"},
26       "host": {"pid": 3848448},
27       "args": [
28         {
29           "name": "pathname",
30           "value": "/opt/aes-beta/flag"
31         },
32         {
33           "name": "syscall_pathname",
34           "value": "flag"
35         }
36       ]
37     },
38     {
39       "event_name": "open",
40       "process": {"uid": 10001, "comm": "cat"},
41       "host": {"pid": 3848448},
42       "args": [
43         {"name": "pathname", "value": "flag"},
44         {
45           "name": "returnValue",
```

```
46         "value": 2,  
47         "label": "fd:2"  
48     }  
49 ]  
50 }  
51 ],  
52 "threat": {  
53     "framework": "MITRE ATT&CK Enterprise",  
54     "version": "v19.1",  
55     "tactics": [  
56         {"id": "TA0002", "name": "Execution"},  
57         {"id": "TA0009", "name": "Collection"}  
58     ],  
59     "techniques": [  
60         {  
61             "id": "T1059.004",  
62             "name": "Unix Shell",  
63             "sub_technique": "Unix Shell"  
64         },  
65         {  
66             "id": "T1005",  
67             "name": "Data from Local System"  
68         }  
69     ],  
70     "data_sources": ["File", "Process"],  
71     "data_components": [  
72         "File Access",  
73         "Process Creation"  
74     ]  
75 }  
76 }  
77
```

Listing 6.1: Selected fields from the JSON alert emitted during exploit run E1.

The first two records confirm the successful executable transitions to `/bin/sh` and `/bin/cat`. The third identifies the resolved flag file at the LSM mediation point, while the fourth records the successful result of the corresponding `open` syscall. The repeated host PID and UID in the listing are consistent with all four observations originating from the same target process. Internally, the detector's `process_tree` strategy also uses the task start time to avoid correlating different process lifetimes that reuse the same numeric PID.

The application and detector oracles agreed in all three exploit runs, but they measure different outcomes. Recovering the synthetic flag demonstrates that the attack reached its intended application effect. The alert instead demonstrates that the four configured kernel observations were received and correlated in the expected order. Successful exploitation does not by itself

guarantee alert generation, and an alert does not replace verification of the application outcome. This distinction is particularly relevant to the client-side validation discrepancy discussed below.

6.5.2 Benign Control and Repeated Runs

The benign interaction was used as a negative control for the configured detector. It exercised the target’s normal encryption workflow and returned the expected ciphertext without invoking the command-execution path associated with flag recovery. Repeating this interaction provided a direct comparison with the exploit runs under the same deployment and monitoring configuration.

Table 6.2 reports the application and detector outcomes of the six retained runs. The detailed contents of the exploit alert were examined in the preceding subsection; the table records only whether the configured sequence completed.

Run	Interaction	Application outcome	Alerts	Sequence outcome
B1	Benign	Expected ciphertext returned	0	No matching sequence completed
B2	Benign	Expected ciphertext returned	0	No matching sequence completed
B3	Benign	Expected ciphertext returned	0	No matching sequence completed
E1	Exploit	Synthetic flag recovered	1	Four-event sequence completed
E2	Exploit	Synthetic flag recovered	1	Four-event sequence completed
E3	Exploit	Synthetic flag recovered	1	Four-event sequence completed

Table 6.2: Application and detector outcomes of the repeated functional runs.

The three benign runs completed without producing an `aes-beta-command-execution` alert. Under the same testbed conditions, each exploit run recovered the synthetic flag and completed the configured detector sequence. These observations show consistent behaviour for the two interactions exercised by the campaign. The benign sample covers one selected application workflow, so it does not provide a basis for estimating a general false-positive rate.

Pod restart deltas remained zero throughout the six retained intervals, and the monitor emitted no matching perf-event loss diagnostic. These health checks indicate that no reported restart or transport-loss condition affected the observed outcomes. Event completeness could not be quantified independently because the experiment did not include a separate producer-side event counter.

6.6 Performance Evaluation

6.6.1 Metrics, Sampling and Benchmark Profiles

The performance campaign reused the exploit workload because it provided a repeatable high-activity interval and an independent functional oracle. Two profiles were evaluated.

- The **filtered** profile enabled exact UID 10001 admission in the common eBPF path.
- The **unfiltered** profile retained the same event, policy, detector and output configuration but disabled that early admission.

After each profile change, the setup allowed a ten-second warm-up before the official run. A nominal 75-second sampler recorded one-second observations, but statistics were calculated only from samples whose host timestamps fell within the actual exploit start and end markers. The primary CPU metric was derived from the monitor container’s cgroup-v1 `cpuacct.usage` counter. For each sampling interval, the increase in accumulated CPU time was divided by the elapsed wall-clock time and expressed as a percentage of one CPU core. A value of 100% therefore represents the equivalent of one fully utilised core, while the project’s 5% threshold corresponds to an average demand of 0.05 core.

Memory was observed through two counters belonging to the same container cgroup. The `memory.usage_in_bytes` value represents the overall memory accounted to the cgroup, whereas `total_rss` from `memory.stat` represents its resident-memory component, including descendant cgroups. The latter is a container-cgroup measurement and should not be interpreted as the RSS reported for the monitor process alone through `/proc`.

Each profile was run once in each of three rounds, and their relative order was varied across the campaign. Every run also retained the flag, alert, loss-diagnostic and restart checks so that a low resource value could not be accepted after a failed workload.

6.6.2 Monitor CPU and Memory Consumption

The UID-filtered configuration consistently required less monitor CPU than the unfiltered configuration. Across the three filtered runs, mean consumption ranged from 0.44% to 0.47% of one CPU core. The aggregate mean was 0.46%, with an SD of 0.02 percentage points. In the unfiltered runs, the corresponding values ranged from 5.28% to 6.43%, producing an aggregate mean of 5.73% and an SD of 0.61 percentage points.

Table 6.3 reports the measurements from each monitored run. The 95th percentile describes short periods of increased CPU activity within a run. Its mean across the three repetitions was 1.00% for the filtered profile and 8.38% for the unfiltered profile.

Profile	Run	Retained samples	Mean CPU (% one core)	P95 CPU (% one core)	Peak memory (MiB)	Peak total_rss (MiB)
UID-filtered	R1	45	0.46	1.01	90.46	36.97
UID-filtered	R2	38	0.47	1.12	91.20	36.09
UID-filtered	R3	46	0.44	0.87	92.17	36.67
Unfiltered	R1	52	6.43	9.54	93.72	40.20
Unfiltered	R2	48	5.48	8.06	110.21	41.24
Unfiltered	R3	52	5.28	7.53	96.20	40.98

Table 6.3: Per-run resource measurements for the monitor-container cgroup.

Peak cgroup memory reached 92.17 MiB with UID filtering and 110.21 MiB without filtering. The highest observed `total_rss` values were 36.97 MiB and 41.24 MiB, respectively. Cgroup memory includes all memory charged to the monitor container, whereas `total_rss` identifies its resident-memory component. The reported peaks describe the observed campaign and may vary with workload duration, event rate and runtime conditions.

Node busy time remained close to 94% in both profiles: 93.99% with UID filtering and 94.49% without it. Mean target-container CPU was 38.87% and 33.64% of one core, respectively. These values characterise a heavily loaded

two-CPU testbed and provide context for the monitor measurements. The filtering comparison itself is derived from CPU time charged to the monitor’s container cgroup.

All six monitored runs preserved the functional outcome: the exploit recovered the synthetic flag and generated one complete four-event alert. Pod restart deltas remained zero, and the retained logs contained no matching perf-event loss diagnostic. The resource measurements can therefore be interpreted alongside a successful and consistent monitored workload.

6.6.3 Effect of Kernel-Side UID Filtering

The project acceptance gate applies to the monitor’s mean CPU consumption and requires the filtered profile to remain below 5% of one core. All three filtered run means satisfied the gate, and their aggregate mean was 0.46%. The corresponding unfiltered mean was 5.73%, above the threshold. The profiles differed by the exact kernel-side UID admission setting while retaining the same scenario and functional checks. The observed difference is therefore consistent with early rejection of non-target activity before event transport and user-space processing, although the sequential laboratory design does not isolate that setting from every source of run-to-run variation.

Using the unrounded run data, the filtered mean was approximately 92.0% lower than the unfiltered mean. This percentage compares monitor-container cgroup CPU and uses the unfiltered mean as its denominator. It is not the total cost of eBPF execution, the total node overhead, or a production capacity estimate.

6.7 Discussion of Results

The functional and performance observations answer complementary questions. The cross-Pod campaign shows that the monitor does not need to share a Pod, network namespace or process namespace with the target to observe its node-kernel activity. The four-stage alert also demonstrates that request, mediation and successful syscall evidence can be preserved together across the implemented pipeline. The result is stronger than observing a single hook, but it remains tied to one known sequence and one same-process correlation path.

The profile comparison quantifies the benefit of applying an exact admission

criterion before event transport. With UID filtering disabled, the complete selected event stream reached user space, where policies and detectors continued to perform their intended semantic selection. Enabling the filter discarded events from other effective UIDs earlier and reduced the measured CPU consumption of the monitor while preserving the alert required by the experiment.

This optimisation depends on prior knowledge of a suitable UID and therefore has a narrower operational scope than user-space policies. A security analyst may not know the relevant process identity in advance, and several processes or containerised workloads may share the same numeric UID.

6.8 Threats to Validity and Current Limitations

Internal validity. The experiment ran sequentially on one busy two-core node, and unmeasured background activity could influence CPU samples. Resource limits were absent, so monitor, target and cluster services could contend.

Construct validity. Monitor-container cgroup CPU measures work charged to that cgroup; it does not include all kernel execution induced by eBPF programs and is therefore not total overhead. Cgroup memory and cgroup `total_rss` capture different memory views. Detector completion measures conformance with one predefined sequence, while three benign controls cannot establish a general false-positive rate or detection accuracy.

External validity. The study uses one Rocky Linux kernel, one Docker-backed Minikube node, one CTF service and one scenario-specific detector. Results may differ with other kernels, attachment availability, container run-times, workload rates and node sizes. All components were co-located, so neither cross-node observation nor cluster-wide correlation was evaluated. The exact UID key was controlled for this workload and should not be generalised to workload identity.

Conclusion validity. Three runs per profile permit a transparent descriptive comparison but do not support strong inferential statistics. High node utilisation and sequential execution make duration and target-CPU differences unsuitable for causal conclusions. The approximately 92.0% value is consequently reported only for the filtered-versus-unfiltered monitor-container cgroup CPU means.

Chapter 7

Conclusions and Future Work

This thesis achieved its stated objective by producing and evaluating a node-local runtime security monitoring prototype for the selected Rocky Linux environment. The implemented pipeline connects bounded kernel observations to typed evidence and deterministic local detection. Its scope is defined by the implemented event domain and by the controlled evaluation presented in Chapter 6.

7.1 Summary of the Work

The central problem was the collection of heterogeneous process and security-related kernel observations while keeping their interpretation outside the collection layer. The resulting architecture assigns constrained data extraction and optional early admission to eBPF. The Go runtime handles record normalisation, monitoring scope, detection and presentation. A shared event contract allows observations produced by different physical hooks to enter the same logical event model.

In kernel space, complementary observation points retain different stages of an operation, including requests, security mediation, committed transitions and returned outcomes. Direct and paired producers use a common bounded construction framework. Selective probe planning activates the producers and support programs required by the chosen logical events. Exact UID

admission can reject observations before full enrichment and transport when a suitable fixed identity is available. CO-RE, explicit layout branches and verifier-visible bounds adapt these mechanisms to the verified Rocky Linux kernel; the compatibility claim remains tied to that target.

In user space, schema-directed decoding converts perf-buffer records into a typed representation shared by the subsequent components. Policies determine the relevant event domain and suppression decisions. Detectors evaluate the admitted evidence, either as individual events or as short ordered sequences held in bounded local state. Alerts preserve the observations responsible for a match and may carry MITRE ATT&CK tactics and techniques as classification metadata. The contribution lies in the integration of these established mechanisms into a compact monitoring pipeline adapted to a constrained enterprise-kernel environment.

7.2 Main Results

The functional campaign established the intended node-local observation boundary. In the single-node Minikube testbed, the monitor, vulnerable target and trigger ran in three separate Pods. All three exploit repetitions recovered the synthetic flag and produced one complete alert containing the configured four-event sequence. The three repetitions of the selected benign interaction returned the expected application response without completing that sequence. These runs demonstrate repeatable same-node, cross-Pod visibility and detection for the controlled scenario. The correlation evidence covers the same-process path exercised by the selected detector; its other process relationships were outside the experiment.

The performance campaign measured the effect of exact UID admission under the same workload. Across three runs, the filtered profile consumed a mean of 0.46% of one CPU core in the monitor-container cgroup, and every run remained below the project’s 5% gate. The unfiltered profile recorded a mean of 5.73%. Using the retained measurements, filtering reduced the mean by approximately 92.0% while preserving flag recovery and the complete alert in every monitored run. The metric represents CPU time charged to the monitor cgroup as a percentage of one core and excludes kernel work charged elsewhere. The result therefore quantifies the user-space benefit of early admission for this scenario, without estimating total eBPF, node or system overhead.

7.3 Limitations

The implemented event surface is broader than the current level of technical assurance. Kernel and Go representations of the event contract are maintained separately, and their agreement relies on review and cross-registry tests. Object-backed tests do not yet cover every loading, attachment, partial-initialisation and cleanup path on the target kernel; collective sequence state has explicit bounds.

The analytical and operational scope remains local. Detection follows deterministic configuration rules, with no learned behavioural baseline or persistent process and cluster graphs. Correlation keys identify local processes, resources and cgroups. The prototype records and reports activity; enforcement is outside its implemented functions.

The evaluation covered one enterprise kernel. The evidence supports the engineering claims made for this controlled environment.

7.4 Future Work

Future development should increase the operational capacity of the prototype while preserving bounded kernel collection and configurable user-space analysis. Five areas have immediate value: scalability, performance, broader low-level filtering, additional host-level hooks and richer fields for the existing event catalogue.

Scalability concerns both the capacity of one agent and its use across several nodes. A local instance must remain responsive as event rates, monitored processes, policies and active detectors increase. Longer stress campaigns and progressively larger detector catalogues can establish these limits. For a multi-node deployment, each agent can retain responsibility for local collection and correlation while exporting versioned events and alerts to an external storage or analysis platform. This arrangement keeps distributed state outside the runtime and gives cluster-level processing a clear interface.

Performance analysis should separate the cost of kernel execution, event transport, decoding, policy evaluation, detector processing and output rendering. Workloads with known event volumes could then measure sustained throughput, perf-buffer loss and the effect of buffer sizing. The resulting profiles would guide work on transport, memory allocation and detector state according to measured bottlenecks.

Low-level filtering currently provides exact UID admission, which is effective when the relevant identity is known in advance. Additional bounded selectors could cover host process and group identifiers, command names, cgroup identifiers and selected event-specific values. Expressive predicates and detector logic would continue to run in user space, preserving flexibility and keeping the collector acceptable to the verifier.

The host event domain can grow through hooks selected from concrete detection and investigation requirements. Candidate areas include additional file modification outcomes, kernel key management, process-isolation changes and other interfaces capable of altering host behaviour. Every new logical event must state the kernel stage it observes and preserve the distinction between a request, a mediation point, a committed transition and a returned outcome.

The existing hooks can also provide more precise evidence. Useful additions include return information for operations currently observed only at entry, richer capability-check context and more complete resource details. Such fields can help a security analyst distinguish operations that currently produce similar records and can reduce ambiguity in detector conditions.

Appendix A

Selected eBPF Process-Monitoring Source Code

The following excerpts are taken from `pkg/ebpf/c/project.bpf.c`. Repeated initialisation and serialisation statements are omitted where indicated; the complete implementation remains in the project repository.

A.1 Process Creation

```
1 SEC("tracepoint/syscalls/sys_enter_clone")
2 int tracepoint__syscalls__sys_enter_clone(sys_enter_clone_ctx_t *ctx)
3 {
4     args_t args = {};
5     args.args[0] = ctx->flags;
6     args.args[1] = ctx->stack;
7     args.args[2] = ctx->parent_tid;
8     args.args[3] = ctx->child_tid;
9     args.args[4] = ctx->tls;
10    return save_args(&args, CLONE);
11 }
12
13 SEC("tracepoint/syscalls/sys_exit_clone")
14 int tracepoint__syscalls__sys_exit_clone(sys_exit_generic_ctx_t *ctx)
15 {
16     args_t args = {};
17     if (load_args(&args, CLONE) != 0)
18         return 0;
19     del_args(CLONE);
```

```
20
21     program_data_t p = {};
22     if (!init_program_data(&p, ctx, CLONE))
23         return 0;
24
25     unsigned long flags = args.args[0];
26     long ret = ctx->ret;
27     save_to_submit_buf(&p.event->args_buf,
28                       &flags, sizeof(unsigned long), 0);
29     /* stack, TID pointers and TLS occupy indexes 1--4. */
30     save_to_submit_buf(&p.event->args_buf,
31                       &ret, sizeof(long), 5);
32     return events_perf_submit(&p);
33 }
```

Listing A.1: Entry and return programs used to construct the `clone` event. Repeated serialisation calls are omitted.

```
1 struct task_struct *parent =
2     (struct task_struct *) ctx->args[0];
3 struct task_struct *child =
4     (struct task_struct *) ctx->args[1];
5
6 u32 parent_pid = get_task_host_tgid(parent);
7 u32 parent_tid = get_task_host_pid(parent);
8 u32 child_pid = get_task_host_tgid(child);
9 u32 child_tid = get_task_host_pid(child);
10 u64 child_start = get_task_start_time(child);
11
12 save_to_submit_buf(&p.event->args_buf,
13                   &parent_pid, sizeof(u32), 0);
14 save_to_submit_buf(&p.event->args_buf,
15                   &parent_tid, sizeof(u32), 1);
16 save_to_submit_buf(&p.event->args_buf,
17                   &child_pid, sizeof(u32), 2);
18 save_to_submit_buf(&p.event->args_buf,
19                   &child_tid, sizeof(u32), 3);
20 save_to_submit_buf(&p.event->args_buf,
21                   &child_start, sizeof(u64), 4);
```

Listing A.2: Parent-child identity collected by `sched_process_fork`.

A.2 Executable Transitions

```
1 SEC("tracepoint/syscalls/sys_enter_execve")
2 int tracepoint__syscalls__sys_enter_execve(sys_enter_execve_ctx_t *ctx)
3 {
4     program_data_t p = {};
5     if (!init_program_data(&p, ctx, EXECVE) || !ctx->pathname)
6         return 0;
7 }
```

```
8     save_str_to_buf(&p.event->args_buf, (void *) ctx->pathname, 0);
9     if (ctx->argv)
10         save_str_arr_to_buf(
11             &p.event->args_buf,
12             (const char __user *const __user *) ctx->argv, 1);
13     return events_perf_submit(&p);
14 }
15
16 SEC("tracepoint/syscalls/sys_enter_execveat")
17 int tracepoint__syscalls__sys_enter_execveat(sys_enter_execveat_ctx_t *ctx)
18 {
19     program_data_t p = {};
20     if (!init_program_data(&p, ctx, EXECVEAT))
21         return 0;
22
23     int dirfd = ctx->dirfd;
24     int flags = ctx->flags;
25     save_to_submit_buf(&p.event->args_buf,
26                       &dirfd, sizeof(int), 0);
27     if (ctx->pathname)
28         save_str_to_buf(&p.event->args_buf, (void *) ctx->pathname, 1);
29     save_to_submit_buf(&p.event->args_buf,
30                       &flags, sizeof(int), 2);
31     if (ctx->argv)
32         save_str_arr_to_buf(
33             &p.event->args_buf,
34             (const char __user *const __user *) ctx->argv, 3);
35     return events_perf_submit(&p);
36 }
```

Listing A.3: Collection of `execve` and `execveat` requests at syscall entry.

```
1  /* security_bprm_check: resolved executable under mediation. */
2  if (!bprm)
3      return 0;
4  struct file *file = BPF_CORE_READ(bprm, file);
5  if (!file)
6      return 0;
7
8  void *file_path = get_path_str(
9      __builtin_preserve_access_index(&file->f_path));
10 if (!file_path)
11     return 0;
12
13 dev_t s_dev = get_dev_from_file(file);
14 unsigned long inode_nr = get_inode_nr_from_file(file);
15 save_str_to_buf(&p.event->args_buf, file_path, 0);
16 save_to_submit_buf(&p.event->args_buf,
17                   &s_dev, sizeof(dev_t), 1);
18 save_to_submit_buf(&p.event->args_buf,
19                   &inode_nr, sizeof(unsigned long), 2);
20 /* filename, argc and envc occupy indexes 3--5. */
21
22 /* sched_process_exec: replacement has completed. */
23 struct linux_binprm *completed_bprm =
```

```
24 (struct linux_binprm *) ctx->args[2];
25 if (!completed_bprm)
26     return 0;
27 const char *filename = BPF_CORE_READ(completed_bprm, filename);
28 if (!filename)
29     return 0;
30
31 save_str_to_buf(&p.event->args_buf, (void *) filename, 0);
32 events_perf_submit(&p);
```

Listing A.4: Resolved binary identity and successful executable transition, abridged from `security_bprm_check` and `sched_process_exec`.

```
1  args_t args = {};
2  if (load_args(&args, PROCESS_EXECUTE_FAILED) != 0)
3      return 0;
4  del_args(PROCESS_EXECUTE_FAILED);
5
6  if (ret >= 0)
7      return 0;
8
9  program_data_t p = {};
10 if (!init_program_data(&p, ctx, PROCESS_EXECUTE_FAILED))
11     return 0;
12
13 int operation = (int) args.args[0];
14 int dirfd = (int) args.args[1];
15 const char *pathname = (const char *) args.args[2];
16 int flags = (int) args.args[3];
17
18 save_to_submit_buf(&p.event->args_buf,
19                  &operation, sizeof(int), 0);
20 save_to_submit_buf(&p.event->args_buf,
21                  &dirfd, sizeof(int), 1);
22 if (pathname)
23     save_str_to_buf(&p.event->args_buf, (void *) pathname, 2);
24 save_to_submit_buf(&p.event->args_buf,
25                  &flags, sizeof(int), 3);
26 save_to_submit_buf(&p.event->args_buf,
27                  &ret, sizeof(long), 4);
28 return events_perf_submit(&p);
```

Listing A.5: Return-side emission of failed executable transitions.

A.3 Credential Transitions

```
1 u32 old_uid = BPF_CORE_READ(old_cred, uid.val);
2 u32 new_uid = BPF_CORE_READ(new_cred, uid.val);
3 u32 old_euid = BPF_CORE_READ(old_cred, euid.val);
4 u32 new_euid = BPF_CORE_READ(new_cred, euid.val);
5 u32 old_suid = BPF_CORE_READ(old_cred, suid.val);
```

```
6 u32 new_suid = BPF_CORE_READ(new_cred, suid.val);
7 u32 old_fsuid = BPF_CORE_READ(old_cred, fsuid.val);
8 u32 new_fsuid = BPF_CORE_READ(new_cred, fsuid.val);
9
10 if (old_uid == new_uid &&
11     old_euid == new_euid &&
12     old_suid == new_suid &&
13     old_fsuid == new_fsuid)
14     return 0;
15
16 save_to_submit_buf(&p.event->args_buf,
17                   &old_uid, sizeof(u32), 0);
18 save_to_submit_buf(&p.event->args_buf,
19                   &new_uid, sizeof(u32), 1);
20 save_to_submit_buf(&p.event->args_buf,
21                   &old_euid, sizeof(u32), 2);
22 save_to_submit_buf(&p.event->args_buf,
23                   &new_euid, sizeof(u32), 3);
24 /* saved and filesystem UIDs occupy indexes 4--7. */
25 save_to_submit_buf(&p.event->args_buf,
26                   &flags, sizeof(int), 8);
27 return events_perf_submit(&p);
```

Listing A.6: Change-only collection in `security_task_fix_setuid`.

Bibliography

- [1] Aqua Security. *Installing Tracee*. Official Tracee documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/install/> (visited on 07/29/2026).
- [2] Aqua Security. *libbpfgo: An eBPF Library for Go Powered by libbpf*. Official project repository. 2026. URL: <https://github.com/aquasecurity/libbpfgo> (visited on 07/27/2026).
- [3] Aqua Security. *Tracee Architecture*. Official architecture documentation for Tracee version 0.15. 2023. URL: <https://aquasecurity.github.io/tracee/v0.15/contributing/architecture/> (visited on 07/29/2026).
- [4] Aqua Security. *Tracee Custom Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/custom/overview/> (visited on 07/29/2026).
- [5] Aqua Security. *Tracee Detector API Reference*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/api-reference/> (visited on 07/27/2026).
- [6] Aqua Security. *Tracee Detectors*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/detectors/> (visited on 07/27/2026).
- [7] Aqua Security. *Tracee Documentation*. 2026. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).
- [8] Aqua Security. *Tracee Documentation Overview*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/overview/> (visited on 07/29/2026).

BIBLIOGRAPHY

- [9] Aqua Security. *Tracee Events*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/events/> (visited on 07/27/2026).
- [10] Aqua Security. *Tracee Output Configuration*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/outputs/> (visited on 07/27/2026).
- [11] Aqua Security. *Tracee Policies: Overview*. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).
- [12] Aqua Security. *Tracee Policy Rules*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/rules/> (visited on 07/29/2026).
- [13] Aqua Security. *Tracee Security Model*. Official Tracee development documentation. 2026. URL: <https://aquasecurity.github.io/tracee/dev/docs/security-model/> (visited on 07/29/2026).
- [14] ECSC2024. *AES-beta Challenge Source*. Official openECSC-2024 challenge repository. 2024. URL: <https://github.com/ECSC2024/openECSC-2024/tree/main/teamitaly-preselection-ctf/pwn01> (visited on 08/13/2026).
- [15] Hojat Gazestani. *Understanding Linux Process UIDs: Real, Effective, Saved, and Filesystem*. Sept. 2, 2025. URL: <https://hojat-gazestani.github.io/blogs/Linux-Standard-UNIX-Access-Control> (visited on 08/04/2026).
- [16] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.
- [17] Elazar Gershuni et al. “Simple and Precise Static Analysis of Untrusted Linux Kernel Extensions”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. Ed. by Kathryn S. McKinley and Kathleen Fisher. PLDI ’19. Association for Computing Machinery, 2019, pp. 1069–1084. DOI: [10.1145/3314221.3314590](https://doi.org/10.1145/3314221.3314590). URL: <https://doi.org/10.1145/3314221.3314590>.
- [18] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Ed. by Joseph A. Calandrino and Carmela Troncoso. Anaheim, CA: USENIX Association, Aug. 2023, pp. 5971–5988. ISBN: 978-1-

BIBLIOGRAPHY

- 939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [19] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. DOI: [10.1109/ACCESS.2025.3605432](https://doi.org/10.1109/ACCESS.2025.3605432). URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [20] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. 2024. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).
- [21] Linux Kernel Community. *eBPF Verifier*. 2026. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).
- [22] Linux man-pages project. *bpf-helpers(7) — List of eBPF Helper Functions*. 2026. URL: <https://man7.org/linux/man-pages/man7/bpf-helpers.7.html> (visited on 07/27/2026).
- [23] Linux man-pages project. *bpf(2) — Perform a Command on an Extended BPF Map or Program*. 2026. URL: <https://man7.org/linux/man-pages/man2/bpf.2.html> (visited on 07/27/2026).
- [24] Linux man-pages project. *inode(7) — Linux Manual Page*. 2026. URL: <https://man7.org/linux/man-pages/man7/inode.7.html> (visited on 08/06/2026).
- [25] Red Hat. *What Is Backporting and How Does It Affect Red Hat Enterprise Linux?* Red Hat Customer Portal. June 2025. URL: <https://access.redhat.com/solutions/57665> (visited on 07/27/2026).
- [26] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [27] Dave Thaler. *BPF Instruction Set Architecture (ISA)*. RFC 9669. Internet Engineering Task Force, Oct. 2024. DOI: [10.17487/RFC9669](https://doi.org/10.17487/RFC9669). URL: <https://www.rfc-editor.org/info/rfc9669>.
- [28] The Cobra Authors. *Cobra: A Commander for Modern Go CLI Interactions*. Official project repository. 2026. URL: <https://github.com/spf13/cobra> (visited on 08/10/2026).
- [29] The Falco Project Authors. *Falco Event Sources*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/> (visited on 07/27/2026).

BIBLIOGRAPHY

- [30] The Falco Project Authors. *Falco Kernel Event Architecture*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/event-sources/kernel/architecture/> (visited on 07/27/2026).
- [31] The Falco Project Authors. *Falco Outputs*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/outputs/> (visited on 07/27/2026).
- [32] The Falco Project Authors. *Falco Rule Conditions*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/conditions/> (visited on 07/27/2026).
- [33] The Falco Project Authors. *Falco Rules*. Official Falco documentation. 2026. URL: <https://falco.org/docs/concepts/rules/> (visited on 07/27/2026).
- [34] The Kubernetes Authors. *DaemonSet*. Official Kubernetes documentation. 2026. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/> (visited on 08/27/2026).
- [35] The Kubernetes Authors. *Linux Kernel Security Constraints for Pods and Containers*. Official Kubernetes documentation. 2024. URL: <https://kubernetes.io/docs/concepts/security/linux-kernel-security-constraints/> (visited on 08/13/2026).
- [36] The Kubernetes Authors. *Pods*. Official Kubernetes documentation. 2026. URL: <https://kubernetes.io/docs/concepts/workloads/pods/> (visited on 08/27/2026).
- [37] The Kubernetes Authors. *Service*. Official Kubernetes documentation. 2026. URL: <https://kubernetes.io/docs/concepts/services-networking/service/> (visited on 08/27/2026).
- [38] The Kubernetes Authors. *Services, Load Balancing, and Networking*. Official Kubernetes documentation. 2026. URL: <https://kubernetes.io/docs/concepts/services-networking/> (visited on 08/13/2026).
- [39] The Kubernetes Authors. *Share Process Namespace between Containers in a Pod*. Official Kubernetes documentation. 2024. URL: <https://kubernetes.io/docs/tasks/configure-pod-container/share-process-namespace/> (visited on 08/13/2026).
- [40] The Linux Kernel Documentation. *BPF Design Q&A*. 2026. URL: https://docs.kernel.org/bpf/bpf_design_QA.html (visited on 07/27/2026).
- [41] The Linux Kernel Documentation. *BPF LLVM Relocations*. 2026. URL: https://docs.kernel.org/bpf/llvm_reloc.html (visited on 07/27/2026).

BIBLIOGRAPHY

- [42] The Linux Kernel Documentation. *BPF Maps*. 2026. URL: <https://docs.kernel.org/bpf/maps.html> (visited on 07/27/2026).
- [43] The Linux Kernel Documentation. *BPF Ring Buffer*. 2026. URL: <https://docs.kernel.org/bpf/ringbuf.html> (visited on 07/27/2026).
- [44] The Linux Kernel Documentation. *BPF Type Format (BTF)*. 2026. URL: <https://docs.kernel.org/bpf/btf.html> (visited on 07/27/2026).
- [45] The Linux Kernel Documentation. *Classic BPF vs eBPF*. Describes execution through architecture-specific JIT compilation and, when JIT compilation is not used, through the `__bpf_prog_run()` interpreter. URL: https://docs.kernel.org/bpf/classic_vs_extended.html (visited on 07/27/2026).
- [46] The Linux Kernel Documentation. *Documentation for `/proc/sys/net/:BPF JIT Controls`*. 2026. URL: <https://docs.kernel.org/admin-guide/sysctl/net.html#bpf-jit-enable> (visited on 07/27/2026).
- [47] The Linux Kernel Documentation. *Kernel Probes (Kprobes)*. 2026. URL: <https://docs.kernel.org/trace/kprobes.html> (visited on 07/27/2026).
- [48] The Linux Kernel Documentation. *libbpf Overview*. 2026. URL: https://docs.kernel.org/bpf/libbpf/libbpf_overview.html (visited on 07/27/2026).
- [49] The Linux Kernel Documentation. *Linux Security Module Development*. 2026. URL: <https://docs.kernel.org/security/lsm-development.html> (visited on 07/27/2026).
- [50] The Linux Kernel Documentation. *Linux Tracing Technologies*. 2026. URL: <https://docs.kernel.org/trace/index.html> (visited on 07/27/2026).
- [51] The Linux Kernel Documentation. *Pathname Lookup*. 2026. URL: <https://docs.kernel.org/filesystems/path-lookup.html> (visited on 08/06/2026).
- [52] The Linux Kernel Documentation. *Perf Ring Buffer*. 2026. URL: https://docs.kernel.org/userspace-api/perf_ring_buffer.html (visited on 07/27/2026).
- [53] The Linux Kernel Documentation. *Program Types and ELF Sections*. 2026. URL: https://docs.kernel.org/bpf/libbpf/program_types.html (visited on 07/27/2026).

BIBLIOGRAPHY

- [54] The Linux Kernel Documentation. *The Linux Kernel API*. Version 6.15.0. Documentation for the `security_file_open` Linux Security Module hook. URL: https://docs.kernel.org/6.15/core-api/kernel-api.html#c.security_file_open (visited on 08/04/2026).
- [55] The Linux Kernel Documentation. *Using the Linux Kernel Tracepoints*. 2026. URL: <https://docs.kernel.org/trace/tracepoints.html> (visited on 07/27/2026).
- [56] The Minikube Authors. *Docker Driver*. Official Minikube documentation. 2024. URL: <https://minikube.sigs.k8s.io/docs/drivers/docker/> (visited on 08/13/2026).
- [57] The Minikube Authors. *Minikube Start*. Official Minikube documentation. 2026. URL: <https://minikube.sigs.k8s.io/docs/start/> (visited on 08/27/2026).
- [58] The Tetragon Authors. *Tetragon Enforcement*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/enforcement/> (visited on 07/27/2026).
- [59] The Tetragon Authors. *Tetragon Events*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/events/> (visited on 07/27/2026).
- [60] The Tetragon Authors. *Tetragon Installation FAQ*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/installation/faq/> (visited on 07/27/2026).
- [61] The Tetragon Authors. *Tetragon Tracing Policies*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/> (visited on 07/27/2026).
- [62] The Tetragon Authors. *Tetragon Tracing Policy Selectors*. Official Tetragon documentation. 2026. URL: <https://tetragon.io/docs/concepts/tracing-policy/selectors/> (visited on 07/27/2026).
- [63] Uber Go. *Zap: Structured, Levelled Logging in Go*. URL: <https://github.com/uber-go/zap> (visited on 07/31/2026).

To my family and friends, for their support.